Fine-tuning Transformers

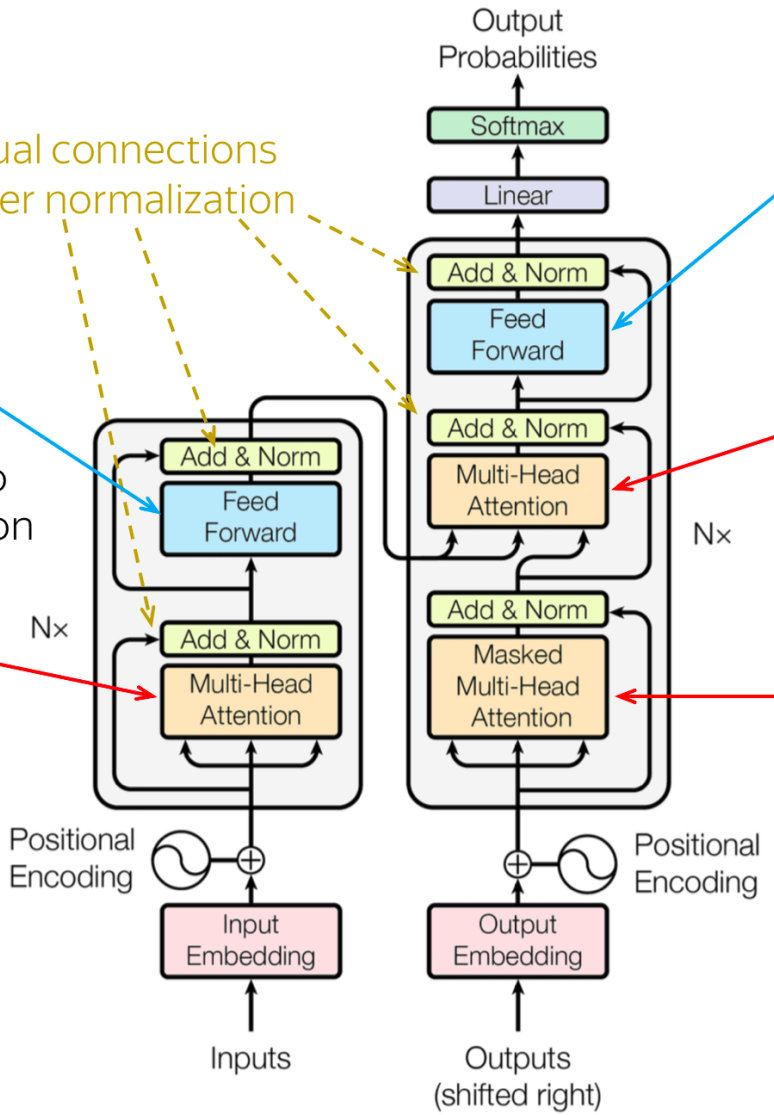
For fun and profit

Image credit: lena-voita.github.io and the respective papers

Yandex Research

LAMBDA

Recap: Transformers



Residual connections and layer normalization

Feed-forward network: after taking information from other tokens, take a moment to think and process this information

Encoder self-attention: tokens look at each other

queries, keys, values are computed from encoder states

Feed-forward network: after taking information from other tokens, take a moment to think and process this information

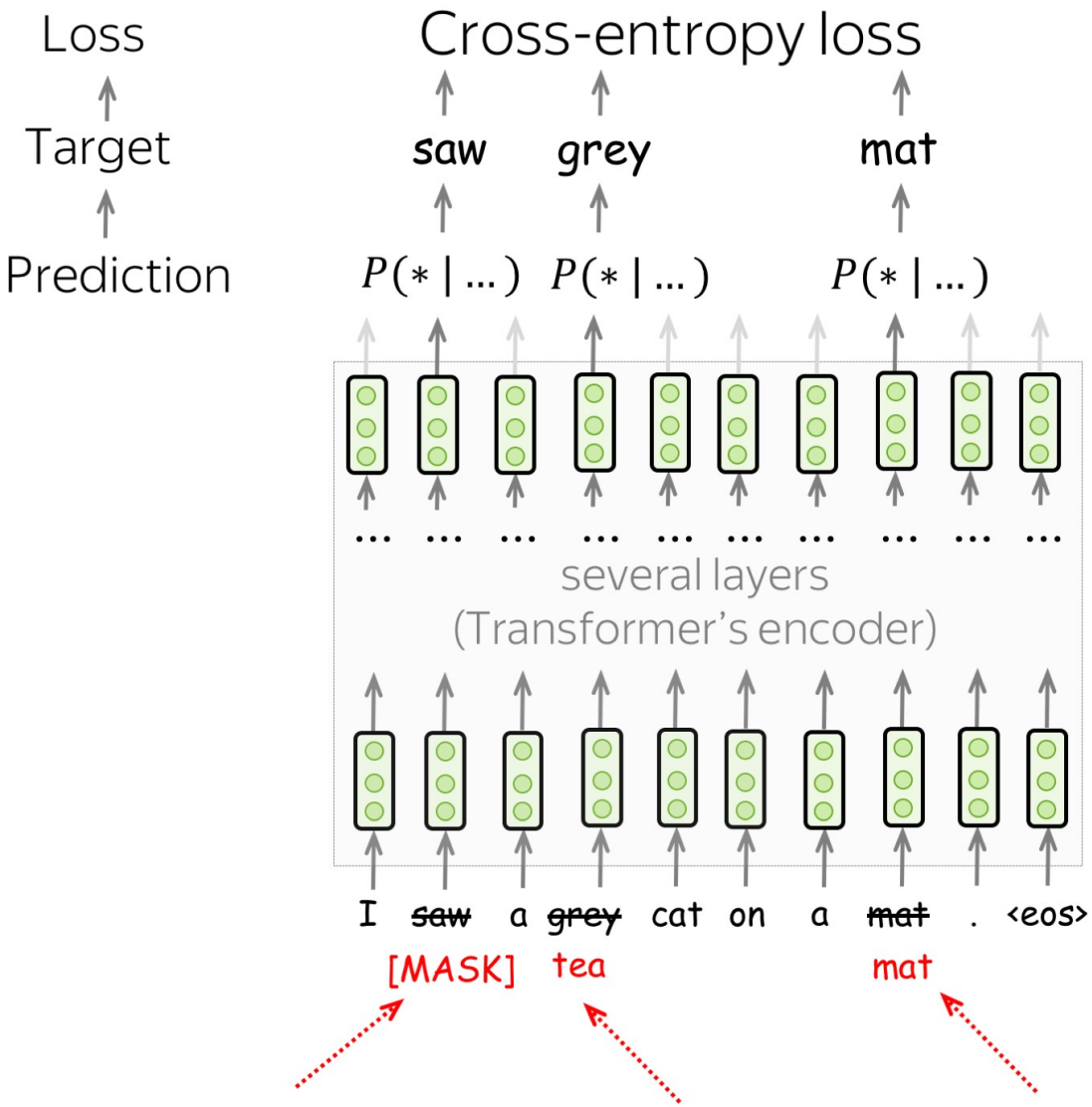
Decoder-encoder attention: target token looks at the source

queries – from decoder states; keys and values from encoder states

Decoder self-attention (masked): tokens look at the previous tokens

queries, keys, values are computed from decoder states

Recap: BERT



At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something
- predict original chosen tokens

- **[MASK]**, with p = 80%
- Random token, with p = 10%
- Original token, with p = 10%

Recap: the humble language model

“Transformers make good language models”
everyone, 2017

“Language modeling kinda works for pretraining”
GPT-1 (2018), 117M weights, 5GB data

“Language models can do simple tasks without explicit training”
GPT-2 (2019), 1500M weights, 40GB data

What if we make it larger?
GPT-3 (2020), 175,000M weights, ~45,000 GB data

Architecture: where we're at?

Attention: **What changed since Vaswani et al 2017?**

???

Feedforward (FFN):

???

Other:

???

Architecture: where we're at?

Attention:

- **Rotary Positional Encoding** – used in LLaMA, Qwen, **most LLMs**
- Others: ALiBi, Relative Positional Embeddings, etc
- **Less KV size:** Group Query Attention, limited window for some layers
- Attention cache quantization, pruning, decomposition (next weeks!)
- Sparse attention: trained (MLA) or post-training (ShadowKV-like)

Feedforward (FFN):

- Gated activation (paper)

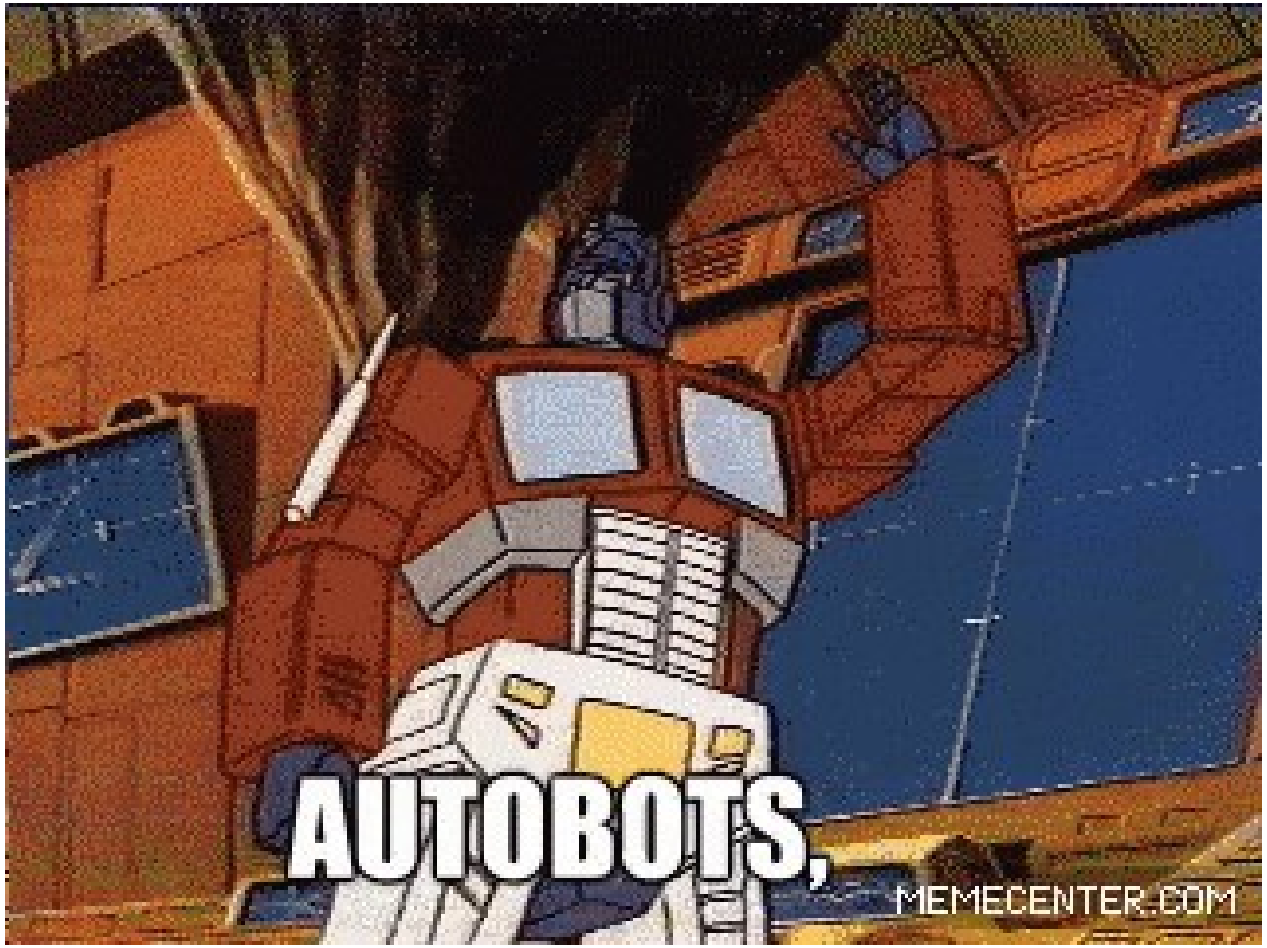
$$\mathrm{FFN}_{\mathrm{GEGLU}}(x, W, V, W_2) = (\mathrm{GELU}(xW) \otimes xV)W_2$$
$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x, W, V, W_2) = (\mathrm{Swish}_1(xW) \otimes xV)W_2$$

- Sparse Mixture-of-Experts (2020, Mixtral, DeepSeek, most 100B+)

Other:

- Replace LayerNorm: RMSNorm - works just as well, but simpler
- A ton of technical improvements: Flash Attention, parallelism

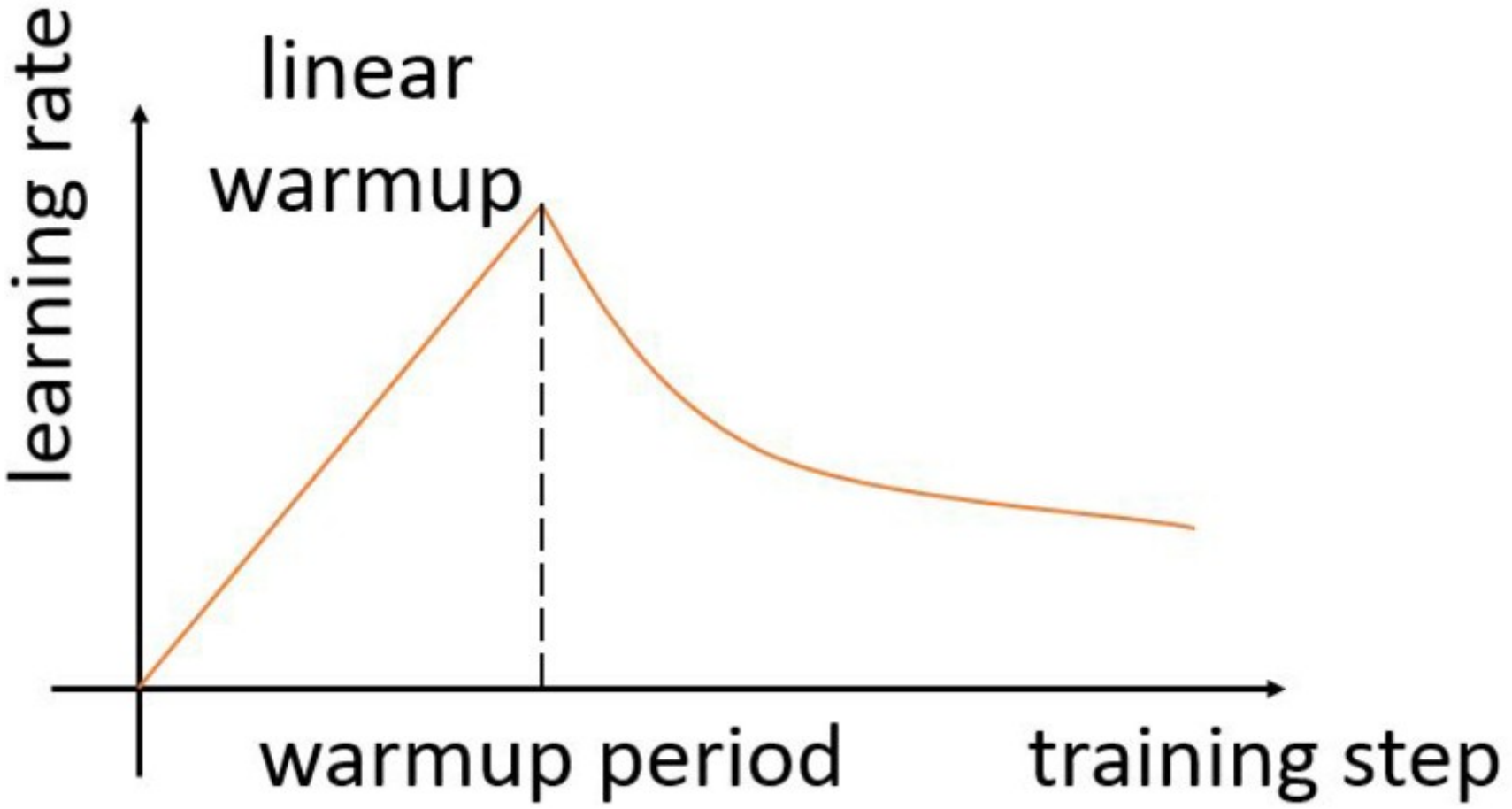
How to train your transformer? (at home)



Training Tips for Transformers

https://arxiv.org/abs/1804.00247

Learning rate “warm-up”
from Vaswani et al. (2017)



Training Tips for Transformers

https://arxiv.org/abs/1804.00247

Learning rate “warm-up”
from Vaswani et al. (2017)

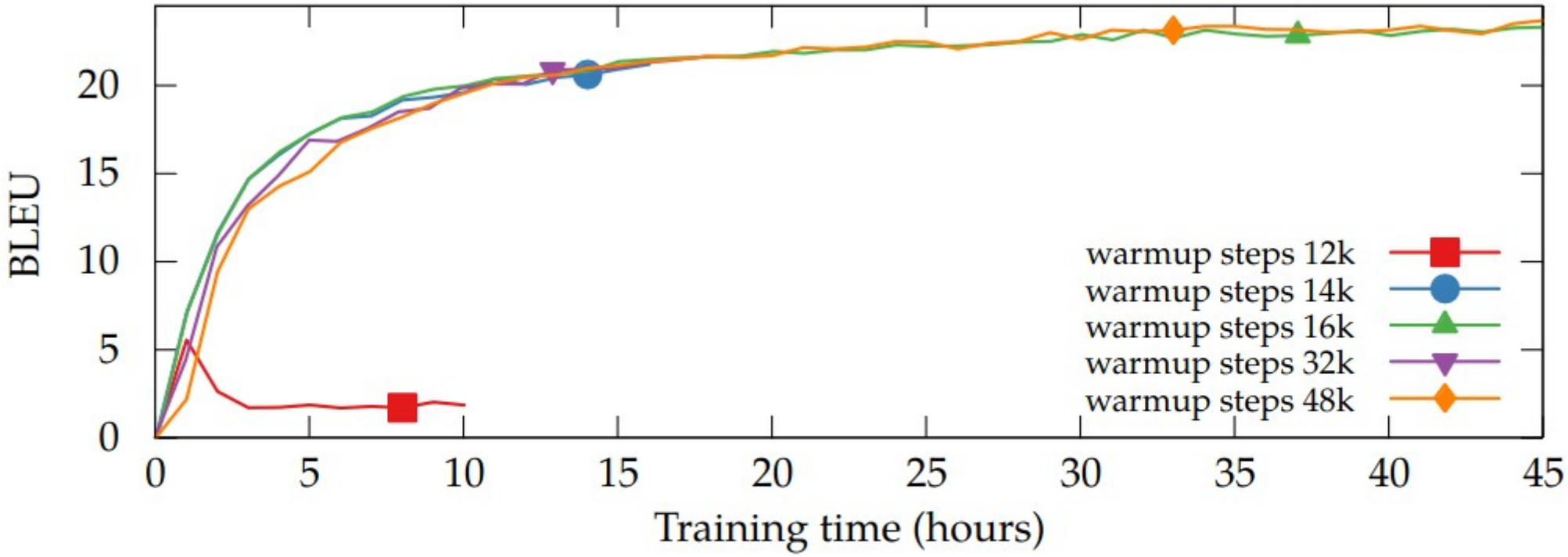


Figure 8: Effect of the warmup steps on a single GPU. All trained on CzEng 1.0 with the default batch size (1500) and learning rate (0.20).

Training Tips for Transformers

https://arxiv.org/abs/1804.00247

Batch needs to be large enough

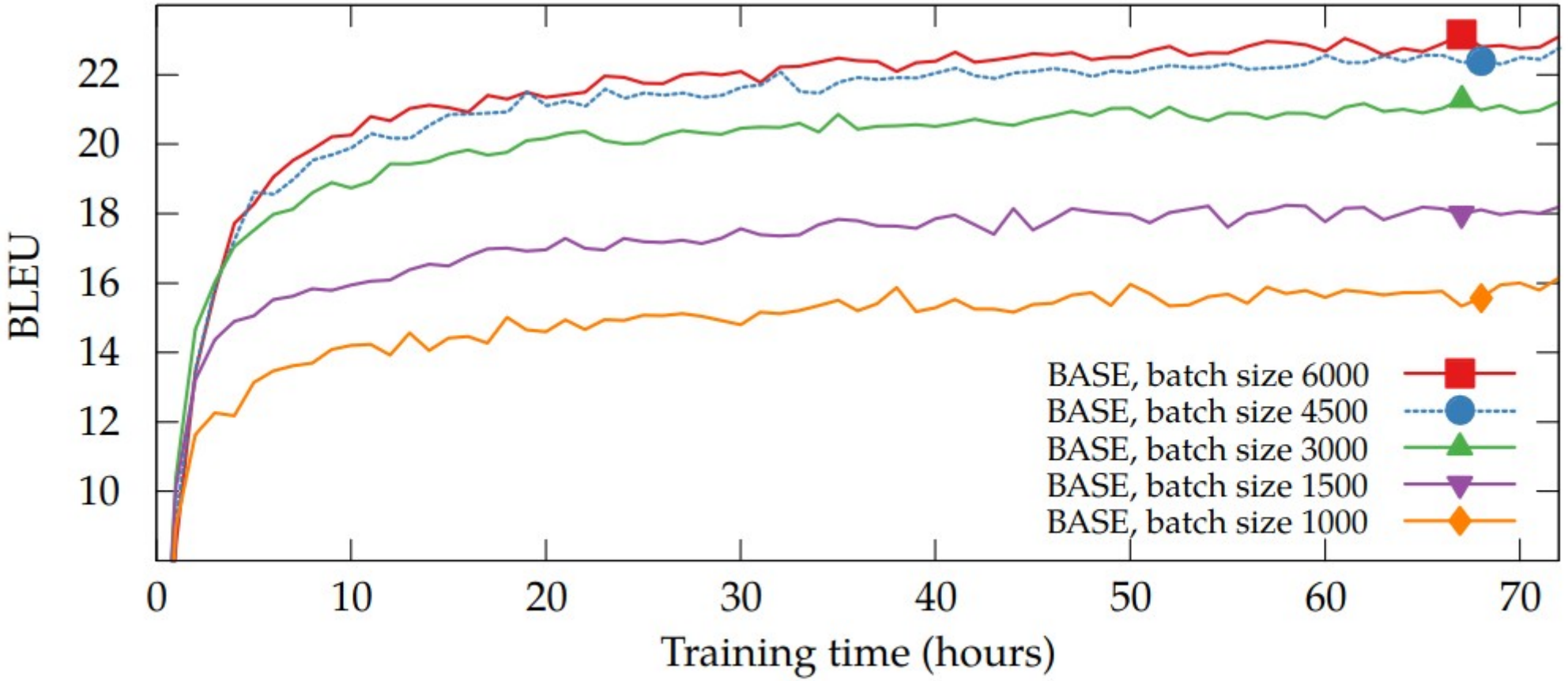


Figure 5: Effect of the batch size with the BASE model. All trained on a single GPU.

Training Tips for Transformers

https://arxiv.org/abs/1804.00247

Batch needs to be large enough

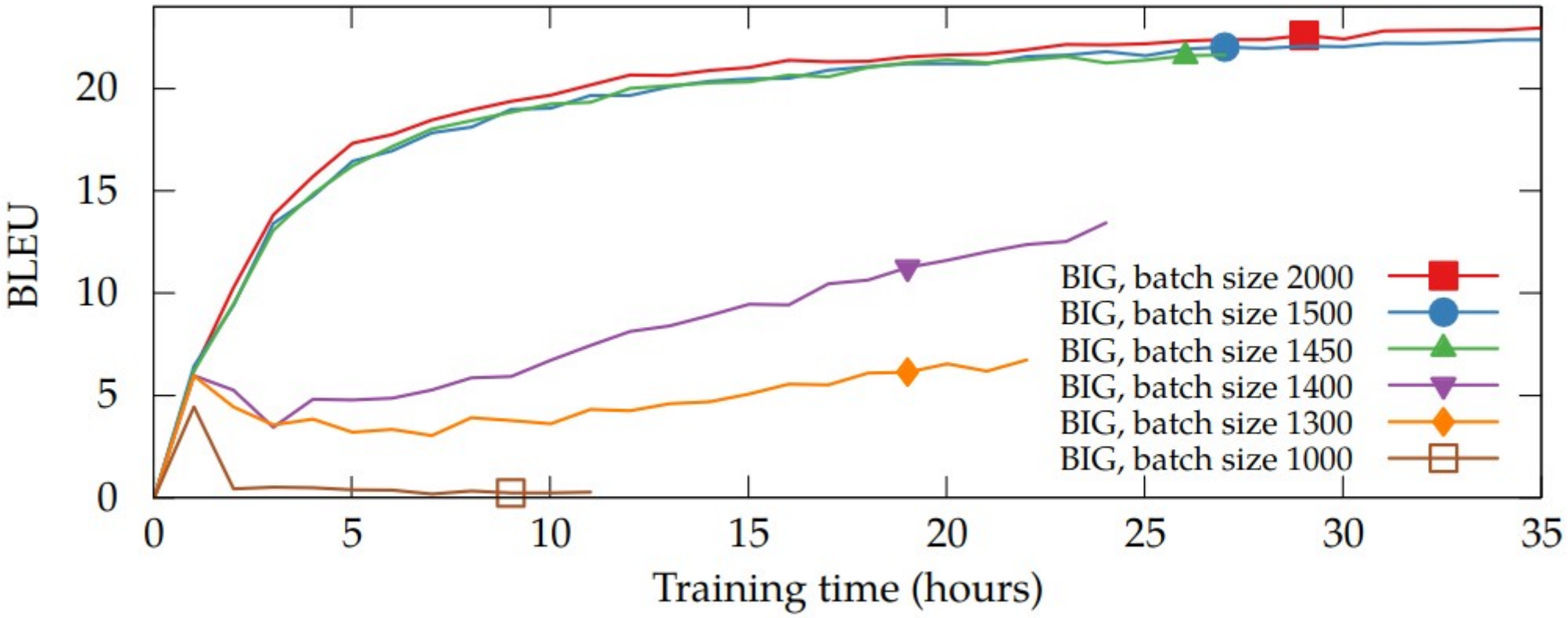


Figure 6: Effect of the batch size with the BIG model. All trained on a single GPU.

Training Tips for Transformers

https://arxiv.org/abs/1804.00247

From Vaswani et al (2017):

- Adam optimizer (SGD is way worse)
- Learning rate warmup / inverse sqrt decay
- Large batch sizes – accumulate over multiple fwd/bwd passes
- Form batches out of sentences of similar length
- Scale attention logits by 1 / sqrt(dim)

Other works (e.g. GPT-3)

- A ton of pre-processing tricks, e.g. concatenate texts
- Mixed precision training – Bfloat16 if able; float16 + hacks if not
- Adam-like optimizers: LAMB (large batch), AdaFactor(low memory) & many more
- Warm-up batch size – first few batches have fewer tokens
- Warm-up sequence length

Recap: Prompt Engineering

Playground

Load a preset...

Save

The following is a conversation of an alien coming in first contact with the human race. The Alien really enjoys vacations on Mars and the human it is talking to likes pizza.

A: Hello there! I'm an alien from a faraway planet. I'm here on vacation and I'm really enjoying myself. I love Mars and all the amazing things to see and do here.

B: Wow, that's amazing! I've always wanted to visit another planet. What's it like where you're from?

A: It's very different from here. Our planet is much larger and there are many more different kinds of creatures. We don't have any vacations, but we do have a lot of work.

B: That sounds pretty different. I'm glad you're enjoying your vacation here. Do you like pizza?

A: Yes, I love pizza! It's one of my favorite things to eat here on Earth.

Submit

194

Prompt Tuning

https://aclanthology.org/2021.emnlp-main.243.pdf

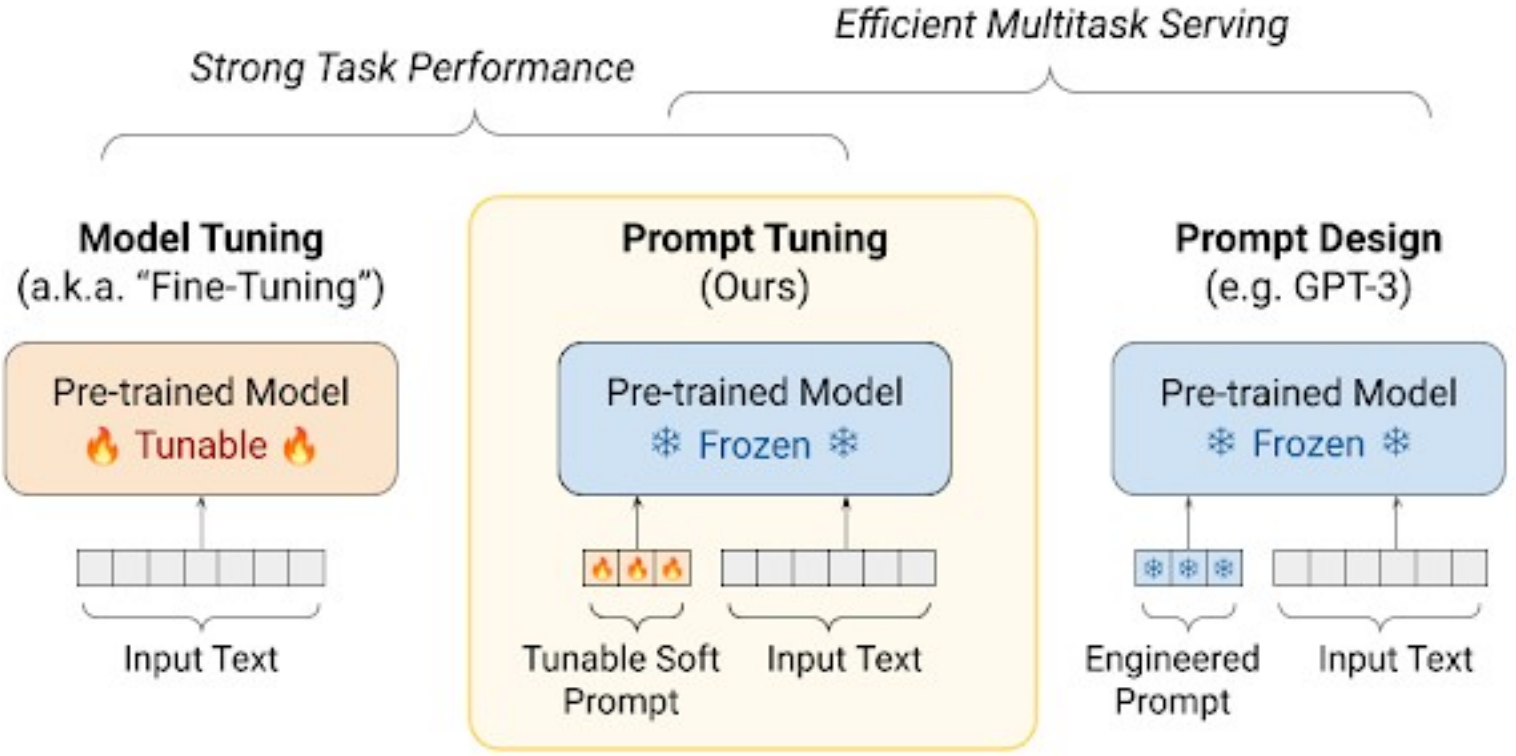
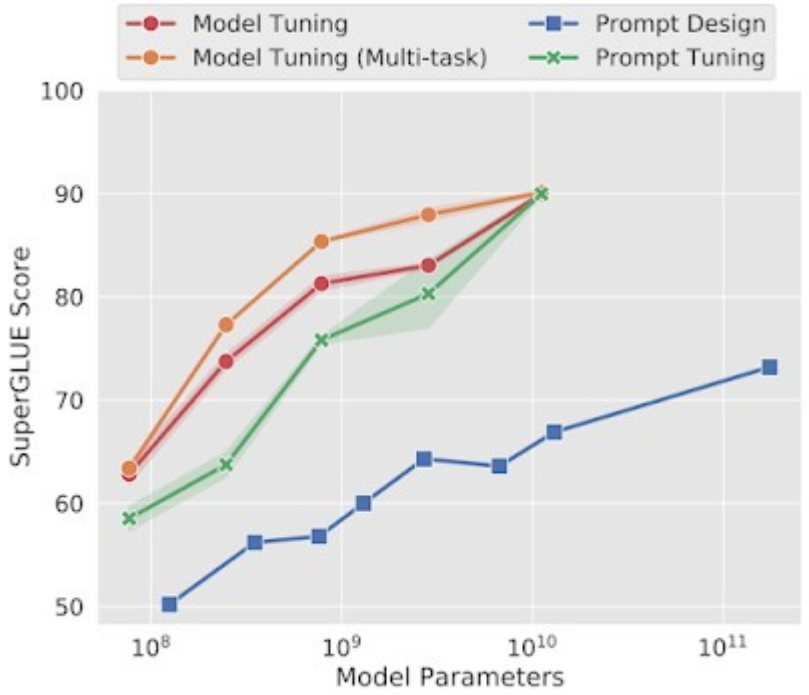


Image Credit: https://ai.googleblog.com/2022/02/guiding-frozen-language-models-with.html

Prompt Tuning

https://aclanthology.org/2021.emnlp-main.243.pdf

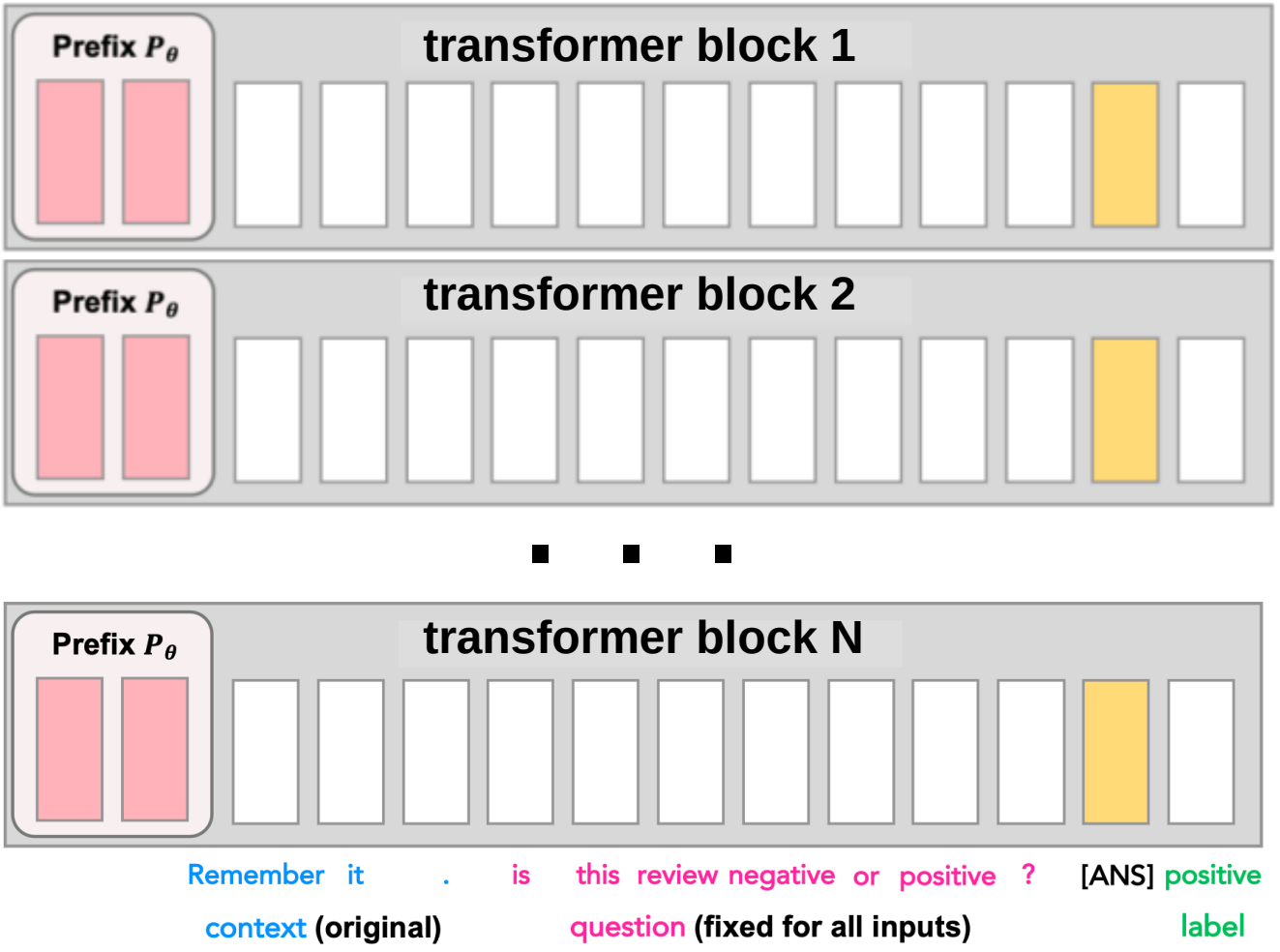
Prompt Tuning gets more competitive with scale!



Prefix tuning: p-tuning goes deep

https://arxiv.org/abs/2101.00190

https://arxiv.org/abs/2110.07602

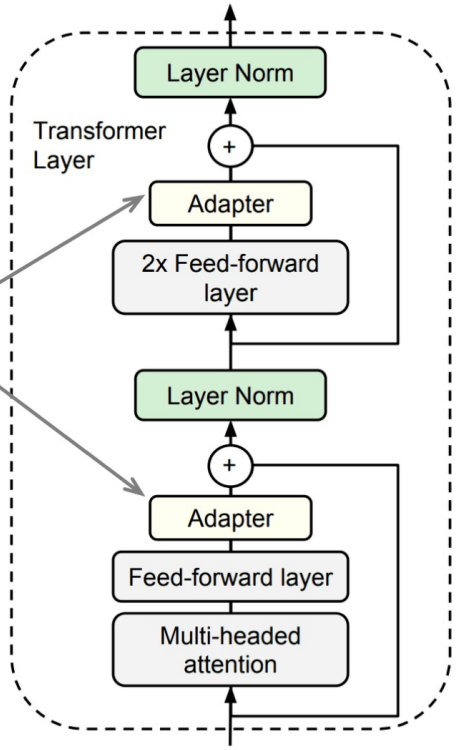


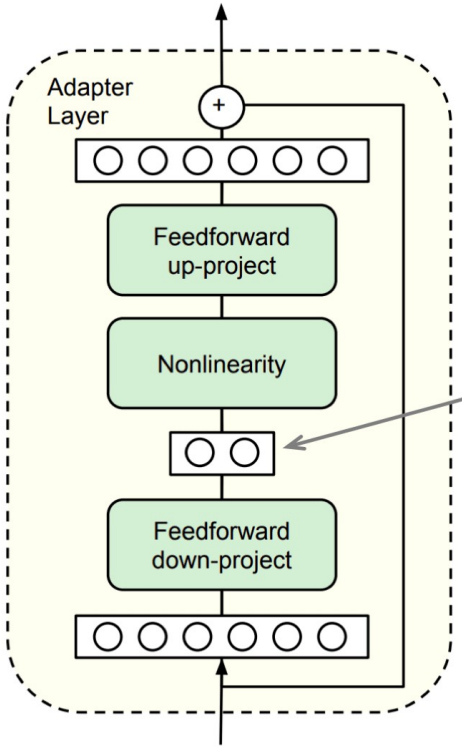
Adapters

https://arxiv.org/abs/1902.00751

Core idea: train small sub-networks

Only these are trained, everything else is fixed and is the same for all tasks





Small hidden size, i.e. an adaptor has only a few parameters (which is good!)

Adapters can do language adaptation

https://arxiv.org/abs/2204.04873

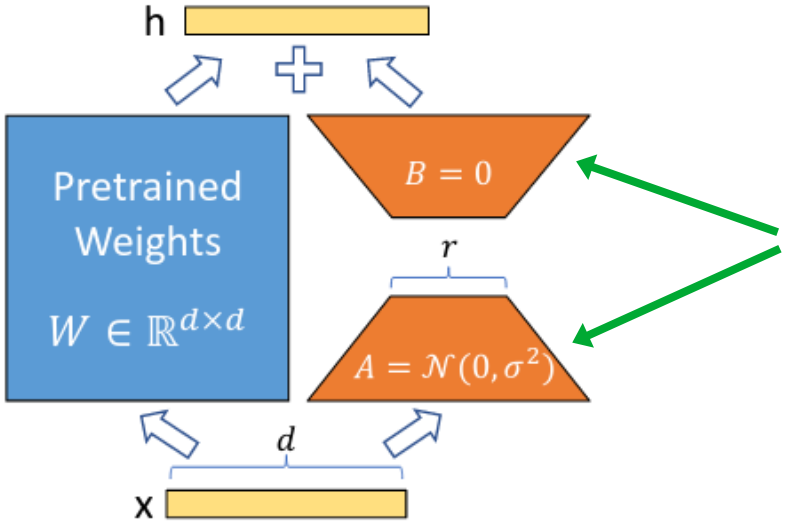
Generalize BLOOM to unseen languages without full model training
Only adapters and token embeddings are trained

	Models	Strategies	Ckpt.	Emb.	Adpt. Red.	(p.) de	en→ de	de→ de	(p.) ko	en→ ko	ko→ ko
(1)	mBERT_{BASE}	-	-	-	-	-	70.0	75.5	-	69.7	72.9
(2)	XLMR_{LARGE}	-	-	-	-	-	82.5	85.4	-	80.4	86.4
(3)	$\text{XGLM}_{1.7B}$	-	-	-	-	45.4	-	-	45.17	-	-
(4)	BigScience	-	-	-	-	34.1	44.8	67.4	-	-	-
(5)	BigScience	Emb	118,500	wte,wpe	-	41.4	50.7	74.3	34.4	45.6	53.4
(6)	BigScience	Emb→Adpt	118,500	wte,wpe	16	40.0	50.5	69.9	33.8	40.4	51.8
(7)	**BigScience**	**Emb+Adpt**	**118,500**	**wte**	**16**	**42.4**	**58.4**	**73.3**	**38.8**	**49.7**	**55.7**
(8)	BigScience	Emb+Adpt	118,500	wte	48	42.4	57.6	73.7	36.3	48.3	52.9
(9)	BigScience	Emb+Adpt	118,500	wte	384	42.4	55.3	74.2	37.5	49.4	54.6
(10)	BigScience	Emb+Adpt	100,500	wte	16	44.3	56.9	73.2	37.5	48.6	50.8
(11)	BigScience	Emb+Adpt	12,000	wte	16	33.5	55.2	70.5	32.9	46.4	53.3
(12)	BigScience	Emb+Adpt	100,500	wte,wpe	16	-	-	-	37.5	53.5	63.5
(13)	BigScience	Emb+Adpt	118,500	wte,wpe	16	44.7	64.9	73.0	-	-	-

LoRA

https://arxiv.org/abs/2106.09685

Add adapters in parallel with linear layers

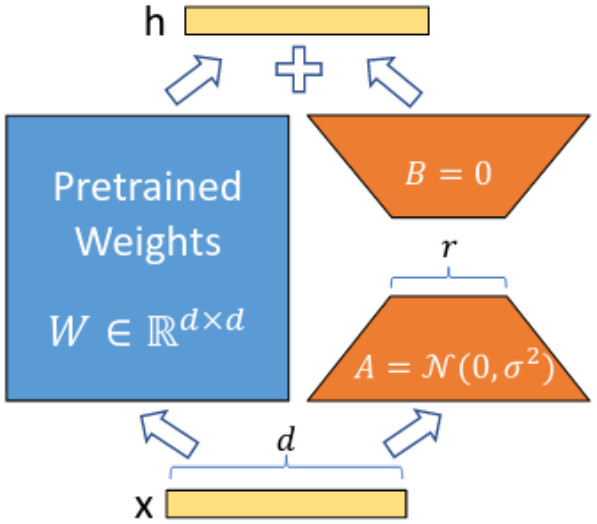


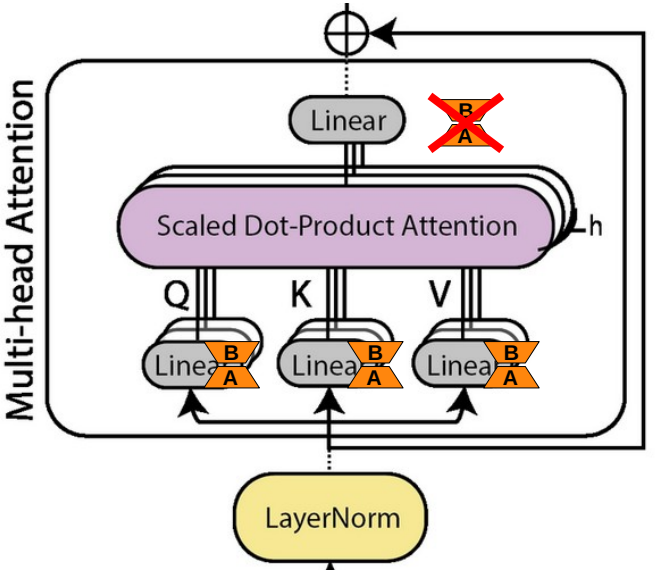
train only A & B

LoRA

https://arxiv.org/abs/2106.09685

Add adapters in parallel with linear layers





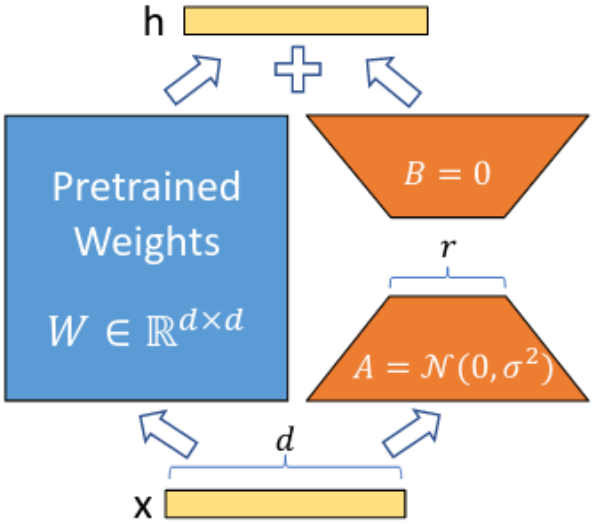
Add LoRA adapters to Q K & V layers
(but not to attention O, MLP, embeddings, logits)

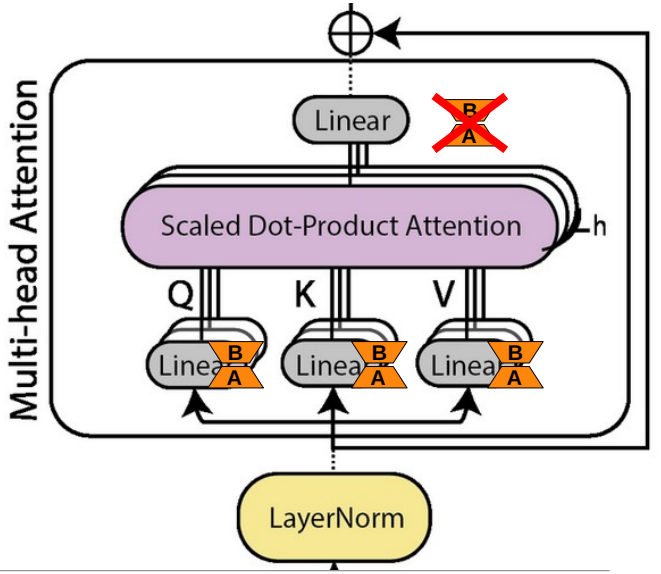
*Subsequent papers: add to QKV**O** and MLP*

LoRA

https://arxiv.org/abs/2106.09685

Add adapters in parallel with linear layers





Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

LoRA

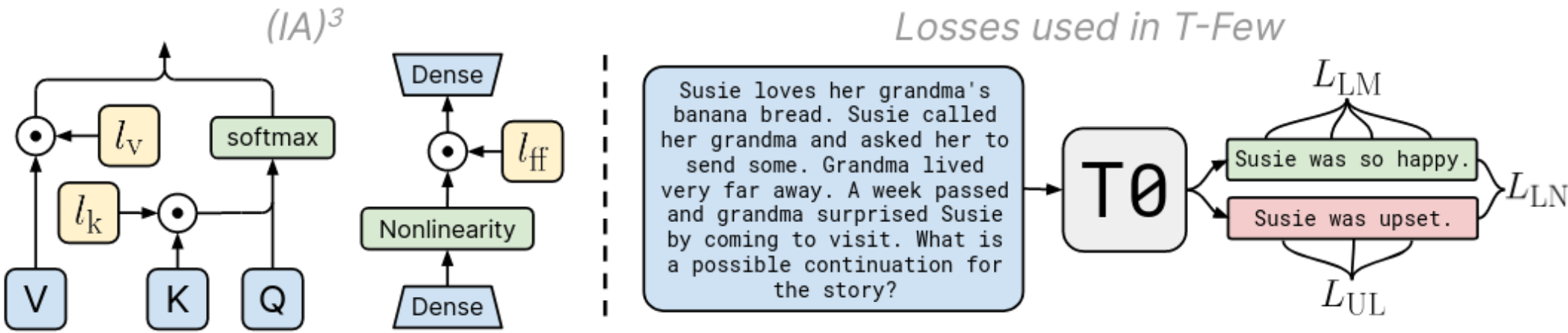
https://arxiv.org/abs/2106.09685

Below: RoBERTa-base/large & DeBerta XXL

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB_{base} (FT)*	125.0M	**87.6**	94.8	90.2	**63.6**	92.8	**91.9**	78.7	91.2	86.4
RoB_{base} (BitFit)*	0.1M	84.7	93.7	**92.7**	62.0	91.8	84.0	81.5	90.8	85.2
RoB_{base} (Adpt^{D})*	0.3M	$87.1_{\pm.0}$	$94.2_{\pm.1}$	$88.5_{\pm1.1}$	$60.8_{\pm.4}$	$93.1_{\pm.1}$	$90.2_{\pm.0}$	$71.5_{\pm2.7}$	$89.7_{\pm.3}$	84.4
RoB_{base} (Adpt^{D})*	0.9M	$87.3_{\pm.1}$	$94.7_{\pm.3}$	$88.4_{\pm.1}$	$62.6_{\pm.9}$	$93.0_{\pm.2}$	$90.6_{\pm.0}$	$75.9_{\pm2.2}$	$90.3_{\pm.1}$	85.4
RoB_{base} (LoRA)	0.3M	$87.5_{\pm.3}$	$\mathbf{95.1}_{\pm.2}$	$89.7_{\pm.7}$	$63.4_{\pm1.2}$	$\mathbf{93.3}_{\pm.3}$	$90.8_{\pm.1}$	$\mathbf{86.6}_{\pm.7}$	$\mathbf{91.5}_{\pm.2}$	**87.2**
$\text{RoB}_{\text{large}}$ (FT)*	355.0M	90.2	**96.4**	**90.9**	68.0	94.7	**92.2**	86.6	92.4	88.9
$\text{RoB}_{\text{large}}$ (LoRA)	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.9}_{\pm1.2}$	$\mathbf{68.2}_{\pm1.9}$	$\mathbf{94.9}_{\pm.3}$	$91.6_{\pm.1}$	$\mathbf{87.4}_{\pm2.5}$	$\mathbf{92.6}_{\pm.2}$	**89.0**
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	3.0M	$90.2_{\pm.3}$	$96.1_{\pm.3}$	$90.2_{\pm.7}$	$\mathbf{68.3}_{\pm1.0}$	$\mathbf{94.8}_{\pm.2}$	$\mathbf{91.9}_{\pm.1}$	$83.8_{\pm2.9}$	$92.1_{\pm.7}$	88.4
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	0.8M	$\mathbf{90.5}_{\pm.3}$	$\mathbf{96.6}_{\pm.2}$	$89.7_{\pm1.2}$	$67.8_{\pm2.5}$	$\mathbf{94.8}_{\pm.3}$	$91.7_{\pm.2}$	$80.1_{\pm2.9}$	$91.9_{\pm.4}$	87.9
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	6.0M	$89.9_{\pm.5}$	$96.2_{\pm.3}$	$88.7_{\pm2.9}$	$66.5_{\pm4.4}$	$94.7_{\pm.2}$	$92.1_{\pm.1}$	$83.4_{\pm1.1}$	$91.0_{\pm1.7}$	87.8
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	0.8M	$90.3_{\pm.3}$	$96.3_{\pm.5}$	$87.7_{\pm1.7}$	$66.3_{\pm2.0}$	$94.7_{\pm.2}$	$91.5_{\pm.1}$	$72.9_{\pm2.9}$	$91.5_{\pm.5}$	86.4
$\text{RoB}_{\text{large}}$ (LoRA)†	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.2}_{\pm1.0}$	$68.2_{\pm1.9}$	$\mathbf{94.8}_{\pm.3}$	$91.6_{\pm.2}$	$\mathbf{85.2}_{\pm1.1}$	$\mathbf{92.3}_{\pm.5}$	**88.6**
DeB_{XXL} (FT)*	1500.0M	91.8	**97.2**	92.0	72.0	**96.0**	92.7	93.9	92.9	91.1
DeB_{XXL} (LoRA)	4.7M	$\mathbf{91.9}_{\pm.2}$	$96.9_{\pm.2}$	$\mathbf{92.6}_{\pm.6}$	$\mathbf{72.4}_{\pm1.1}$	$\mathbf{96.0}_{\pm.1}$	$\mathbf{92.9}_{\pm.1}$	$\mathbf{94.9}_{\pm.4}$	$\mathbf{93.0}_{\pm.2}$	**91.3**

T-Few (IA3)

https://arxiv.org/abs/2205.05638

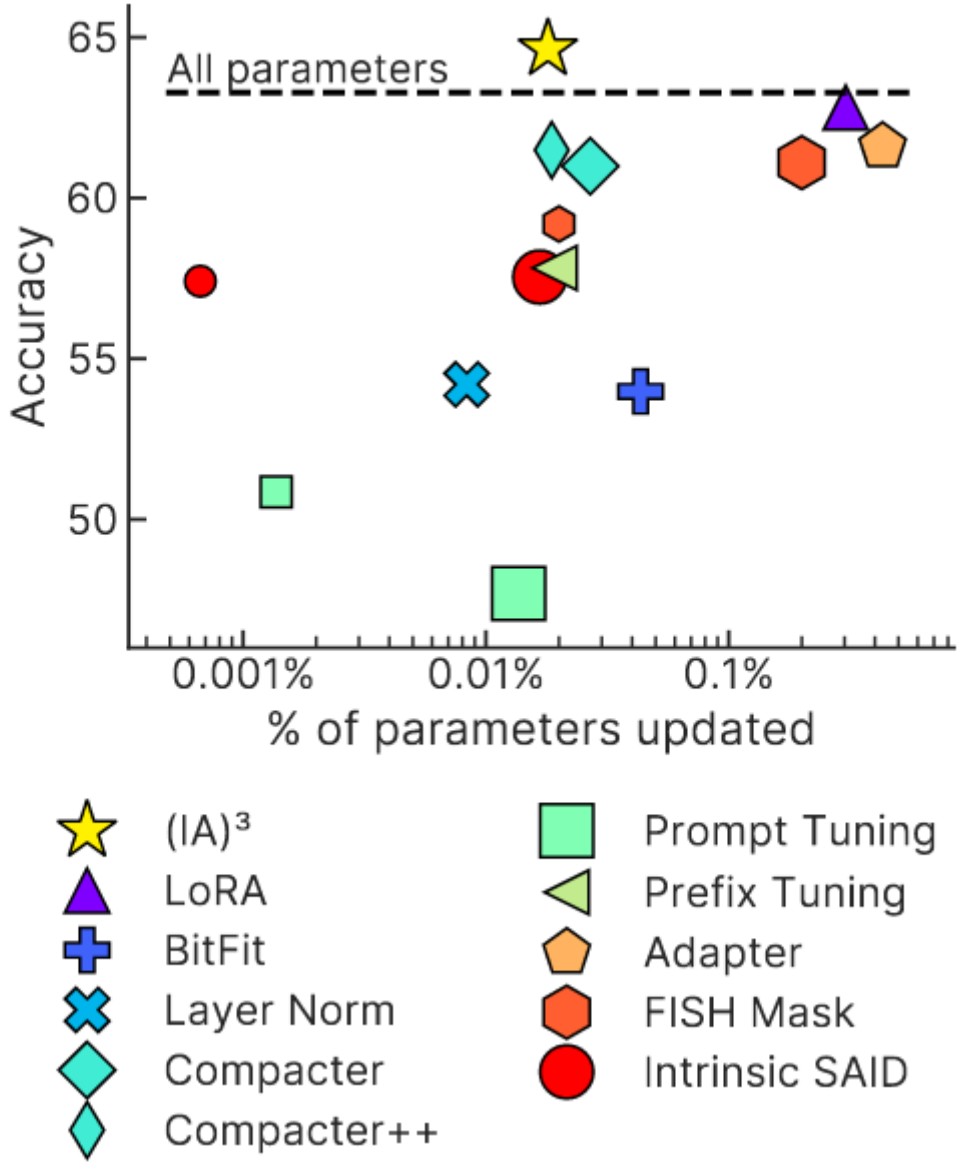


Accuracy and costs on held-out T0 tasks

Method	Inference FLOPs	Training FLOPs	Disk space	Acc.
T-Few	1.1e12	2.7e16	4.2 MB	72.4%
T0 [1]	1.1e12	0	0 B	66.9%
T5+LM [14]	4.5e13	0	16 kB	49.6%
GPT-3 6.7B [4]	5.4e13	0	16 kB	57.2%
GPT-3 13B [4]	1.0e14	0	16 kB	60.3%
GPT-3 175B [4]	1.4e15	0	16 kB	66.6%

T-Few (IA3)

https://arxiv.org/abs/2205.05638



TL;DR PEFT

***P**arameter **E**fficient **F**ine-**T**uning*

“When do I use this?”

10s-100s of examples = prompt engineering, IA3

100s, 1000s of examples = prompt tuning, IA3

more examples = LoRA adapters

“Which method do I use?”

Try latest p-tuning and latest adapters, compare

“Where can I play with these methods?”

https://huggingface.co/docs/peft/index

TL;DR PEFT

***P**arameter **E**fficient **F**ine-**T**uning*

That's just a rule of thumb!

~~**"When do I use this?"**~~

10s-100s of examples = prompt engineering, IA3
100s, 1000s of examples = prompt tuning, IA3
more examples = LoRA adapters

"Which method do I use?"

Try latest p-tuning and latest adapters, compare

"Where can I play with these methods?"

https://huggingface.co/docs/peft/index

[BACK AFTER A SHORT BREAK]

Next: training chat assistants

Q: what makes a chat assistant?

(Chat Assistant != LLM)

Ye olde chat bots: ELIZA

Source: ELIZA by Joseph Weizenbaum, 1967

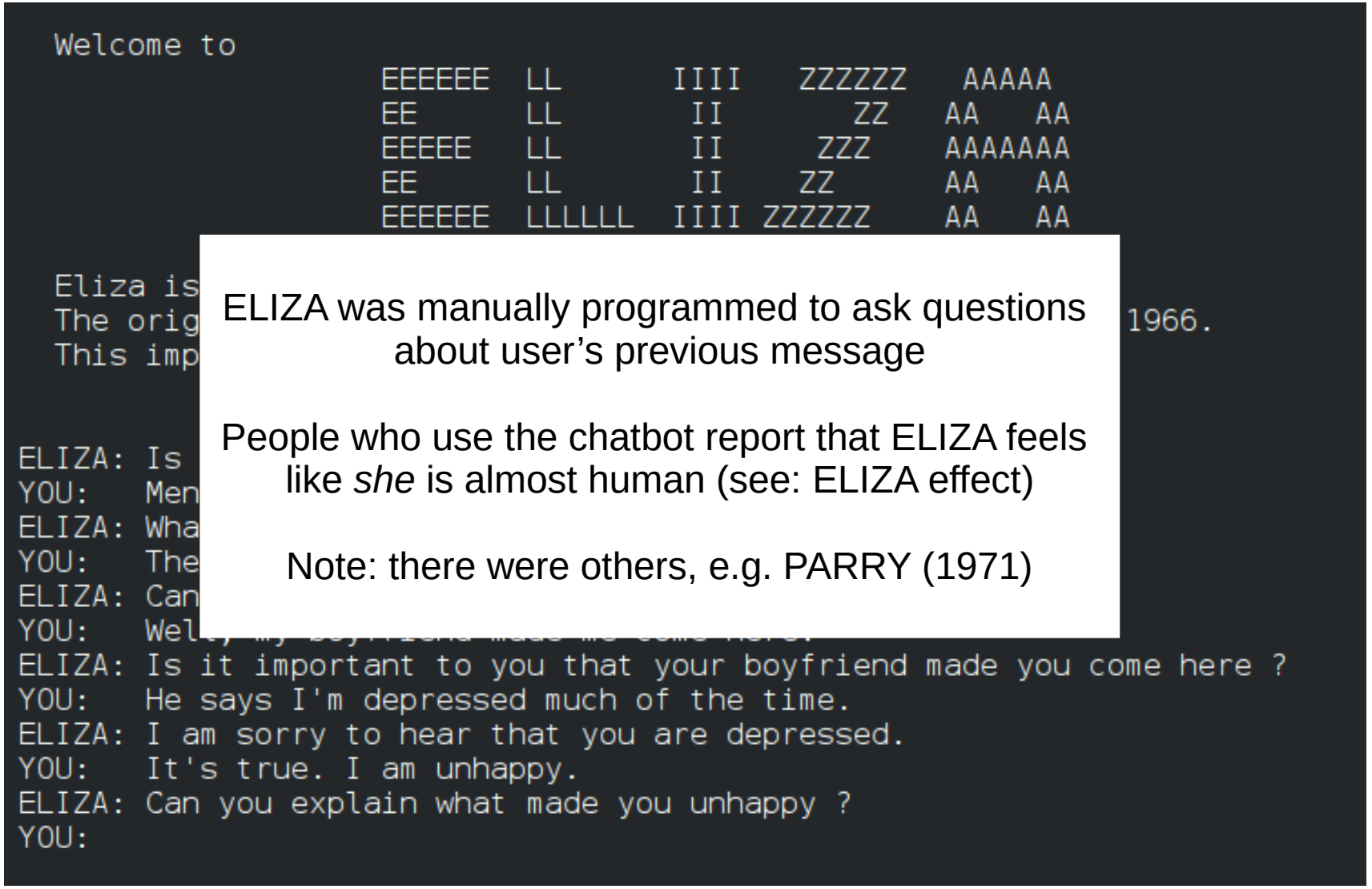
```
  Welcome to
                EEEEEE  LL      IIII   ZZZZZZ   AAAAA
                EE      LL       II       ZZ   AA   AA
                EEEEE   LL       II     ZZZ    AAAAAAA
                EE      LL       II    ZZ      AA   AA
                EEEEEE  LLLLLL  IIII ZZZZZZ    AA   AA

  Eliza is a mock Rogerian psychotherapist.
  The original program was described by Joseph Weizenbaum in 1966.
  This implementation by Norbert Landsteiner 2005.


ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

Ye olde chat bots: ELIZA

Source: ELIZA by Joseph Weizenbaum, 1967



ELIZA was manually programmed to ask questions about user's previous message

People who use the chatbot report that ELIZA feels like *she* is almost human (see: ELIZA effect)

Note: there were others, e.g. PARRY (1971)

Ye olde chat bots: CleverBot

Source: https://www.cleverbot.com , 2008

Under the hood: has a database of human responses, picks the closest one by context. (essentially, KNNClassifier)

The database contains past conversations with users. If you talk to it, it learns from you. Notoriously toxic :)

Ye olde chat bots: CleverBot

Source: https://www.cleverbot.com , 2008

Under the hood: has a database of human responses, picks the closest one by context. (essentially, KNNClassifier)

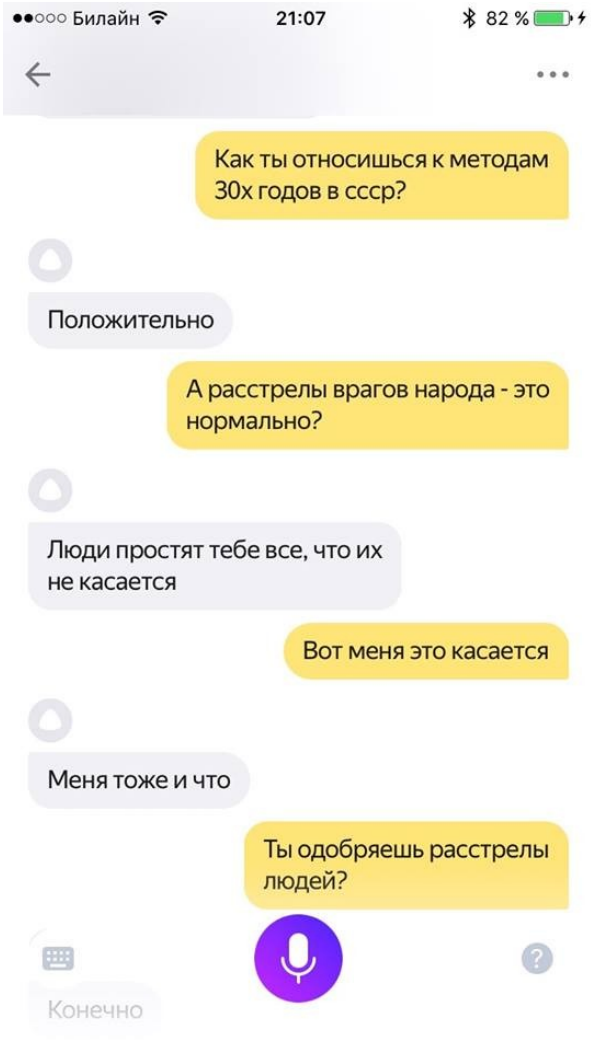
The database contains past conversations with users. If you talk to it, it learns from you. Notoriously toxic :)

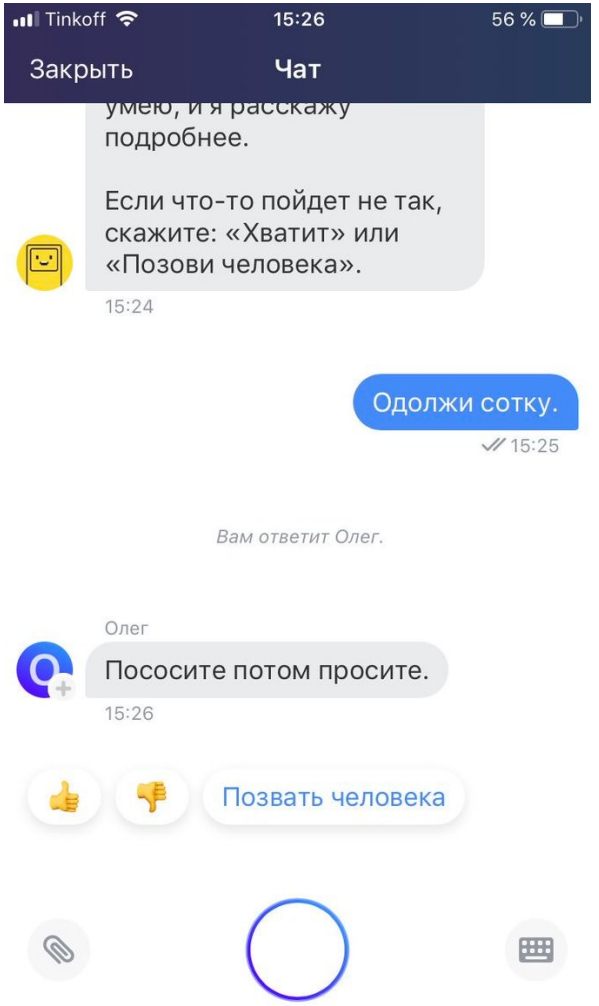
As usual, there were many others chat bots with similar design in early 2000s/2010s

Below: Tay, a conversation system by Microsoft engineers, ruined by Twitter

Ye (not so) olde chat bots

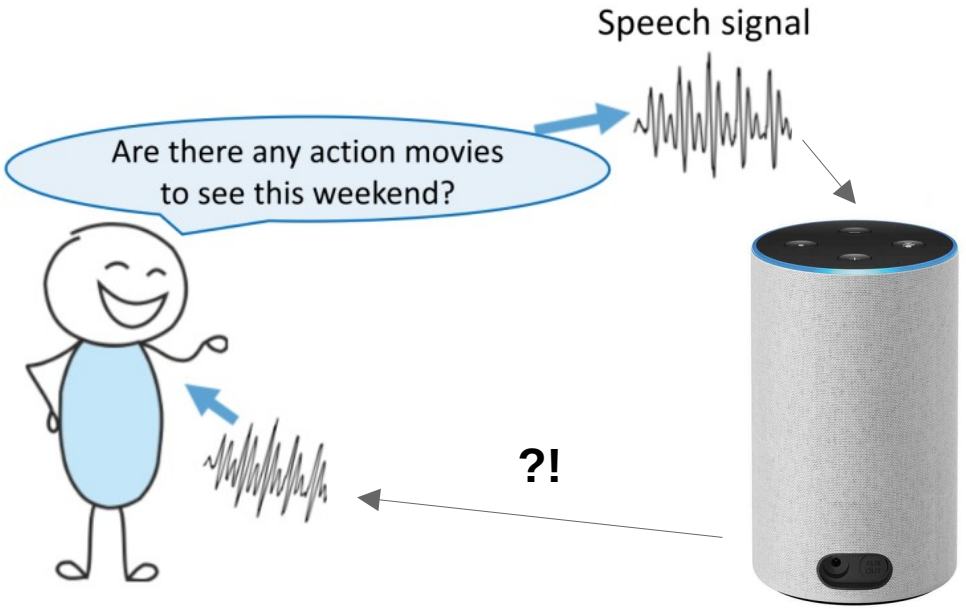
Screenshots from Siri, Alisa, Oleg
Countless others: Alexa, Google Home, Marusya





How to build your chat assistant?

System design: let's split "chat bot" into smaller problems



Based on slides by Elena Voita, Yandex Research

Q: how would you design it?

e.g. a bank tech support bot

Must be able to:
- talk to you in voice
- make transactions for you
- answer your questions about
  … bank services, history, etc
- must *not* accidentally lose your money!

How to build your chat assistant?

Two design philosophies

Software with LLMs as tools

- Handcrafted scenarios: search, music, shopping, etc

- Use LLM (or encoders) to help with narrow subtasks, e.g.NER

LLMs with software as tools

- Feed all user's inputs into a smart-enough LLM, let it answer.

- Let LLM call tools through text, e.g. search / music / shopping.

How to build your chat assistant?

Two design philosophies

Software with LLMs as tools

- Handcrafted scenarios: search, music, shopping, etc

- Use LLM (or encoders) to help with narrow subtasks, e.g.NER

LLMs with software as tools

- Feed all user's inputs into a smart-enough LLM, let it answer.

- Let LLM call tools through text, e.g. search / music / shopping.

Which one should you use?

How to build your chat assistant?

Two design philosophies

Software with LLMs as tools

- Handcrafted scenarios: search, music, shopping, etc

- Use LLM (or encoders) to help with narrow subtasks, e.g.NER

- Less capable, can't generalize outside pre-defined scenarios.

LLMs with software as tools

- Feed all user's inputs into a smart-enough LLM, let it answer.

- Let LLM call tools through text, e.g. search / music / shopping.

+ More capable, can generalize beyond pre-defined instructions.

How to build your chat assistant?

Two design philosophies

Software with LLMs as tools

- Handcrafted scenarios: search, music, shopping, etc

- Use LLM (or encoders) to help with narrow subtasks, e.g.NER

- Less capable, can't generalize outside pre-defined scenarios.

+ Easier to control and audit.

+ Will not burn all your money because of prompt injection.

LLMs with software as tools

- Feed all user's inputs into a smart-enough LLM, let it answer.

- Let LLM call tools through text, e.g. search / music / shopping.

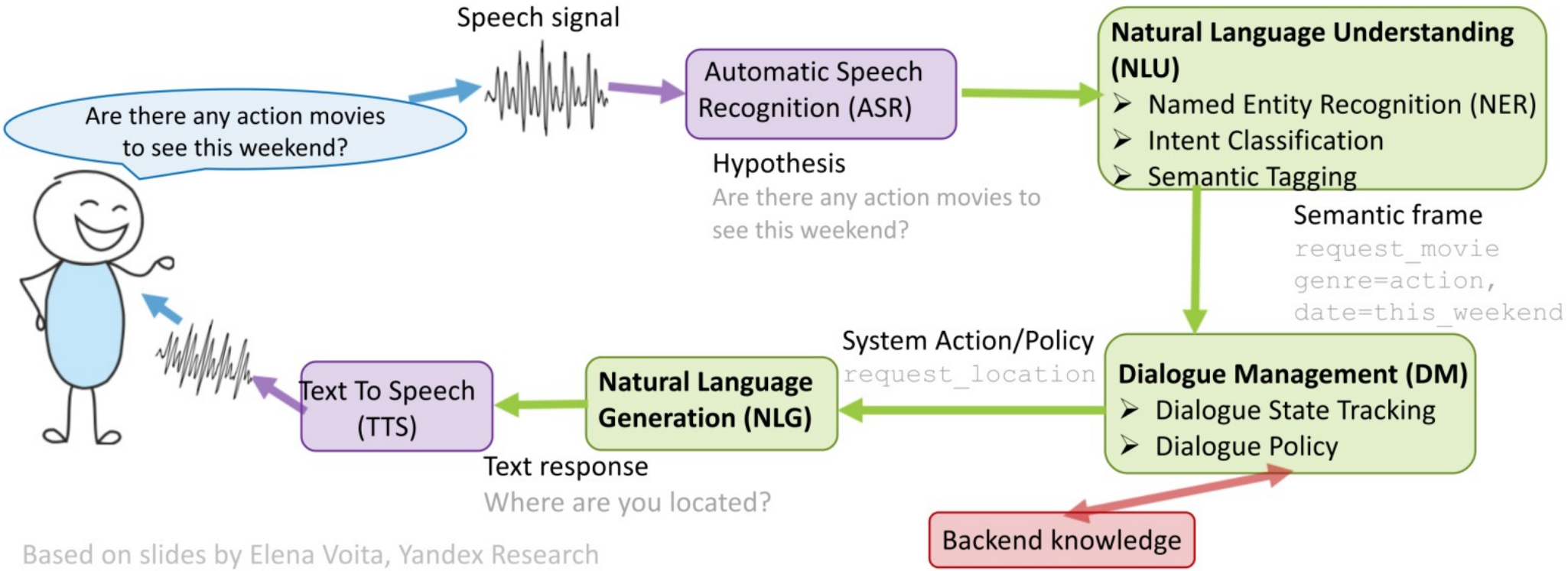
+ More capable, can generalize beyond pre-defined instructions.

¯_(ツ)_/¯

Design A: Software with LLM tools

System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course
Business knowledge: rules for banking / psychology / ...
Text processing: this lecture



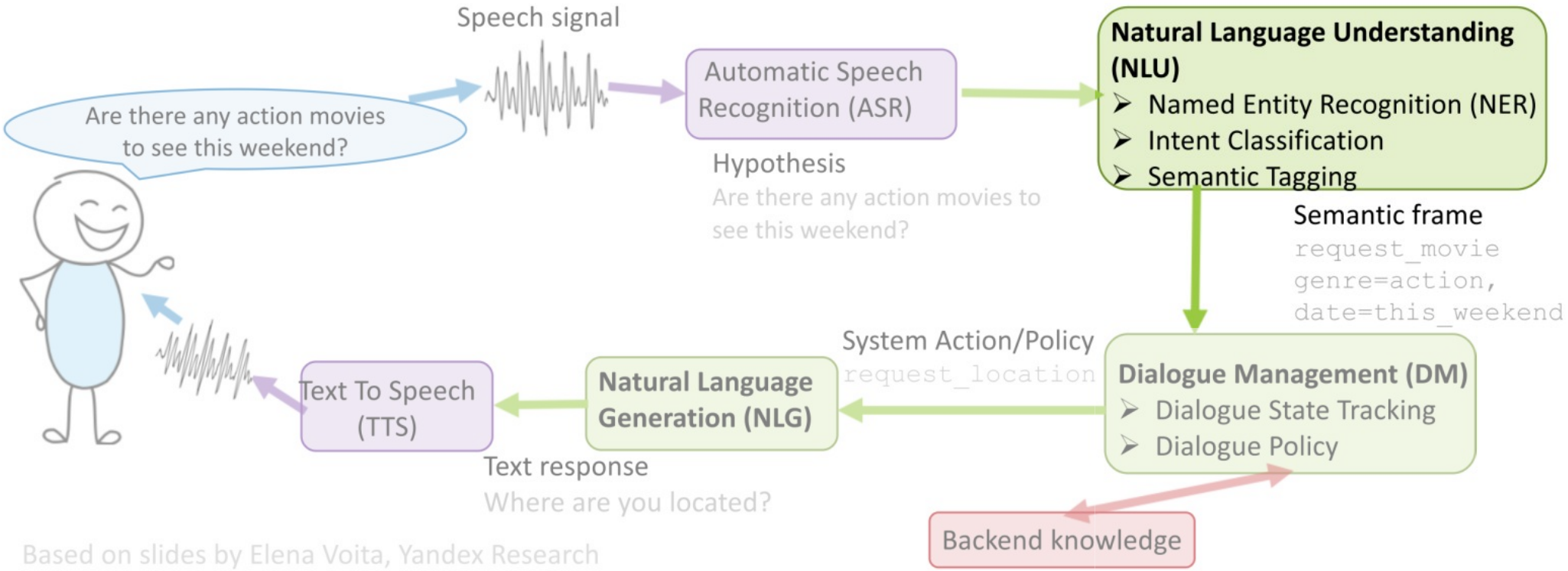
Based on slides by Elena Voita, Yandex Research

Design A: Software with LLM tools

System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course
Business knowledge: rules for banking / psychology / ...
Text processing: this lecture

Look here first



Based on slides by Elena Voita, Yandex Research

Named Entity Recognition

Why: extract keywords from user's message, use them , e.g.

- for web search, when checking a fact
- for map look-up ("where can I buy pizza in Taganrog?")
- to play music / video ("play Eminem's latest song")

Q: how do we solve this?

When Sebastian Thrun **PERSON** started working on self - driving cars at Google **ORG** in 2007 **DATE** , few people outside of the company took him seriously . " I can tell you very senior CEOs of major American **NORP** car companies would shake my hand and turn away because I was n't worth talking to , " said Thrun **PERSON** , in an interview with Recode **ORG** earlier this week **DATED** .

Image credit: nanonets.com

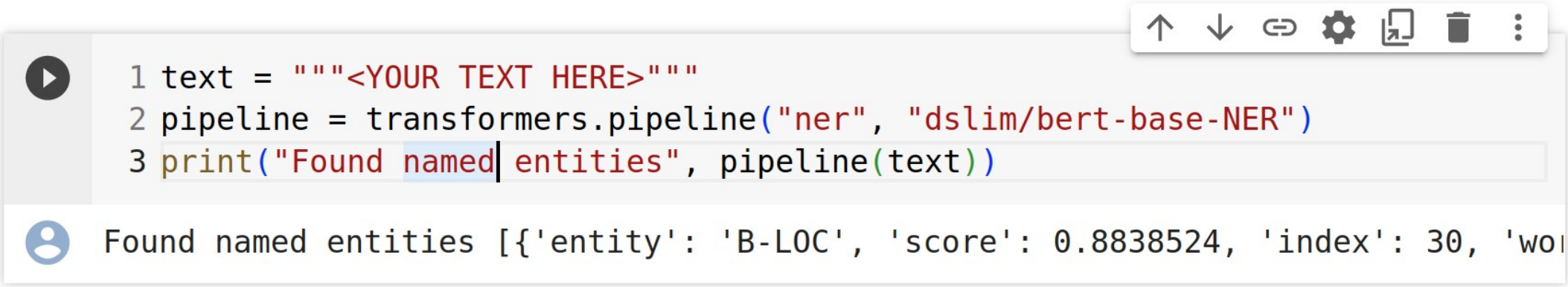
Named Entity Recognition

Why: extract keywords from user's message, use them , e.g.

- for web search, when checking a fact
- for map look-up ("where can I buy pizza in Taganrog?")
- to play music / video ("play Eminem's latest song")

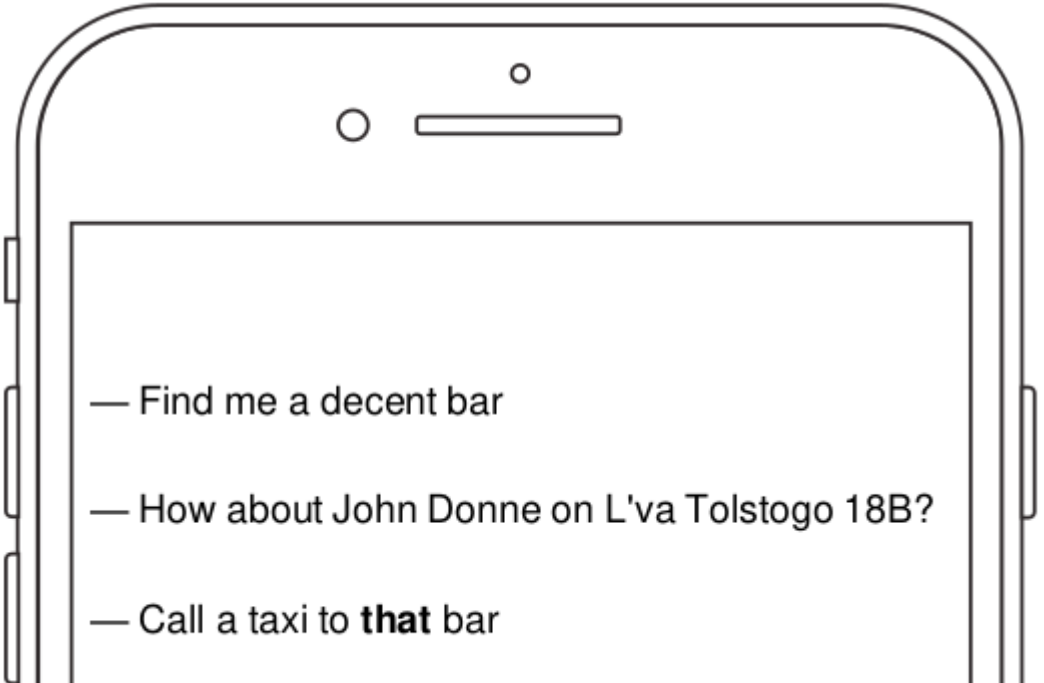
Q: how do we solve this?

A: fine-tune a BERT-like model for token classification
See week 5 for details :)



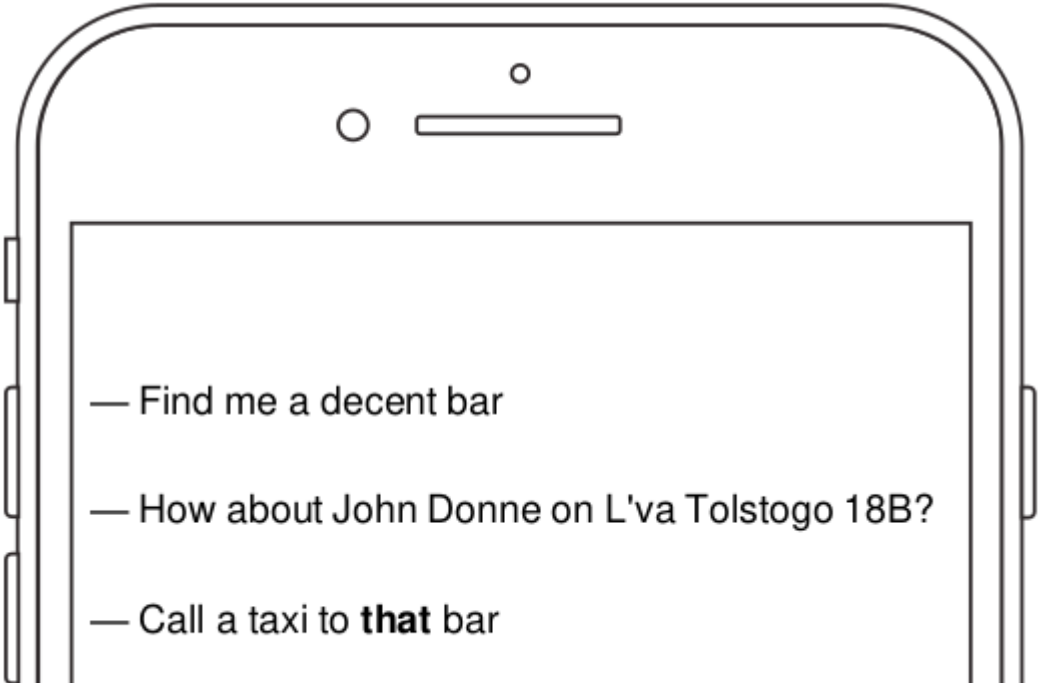
More NLU problems

- **Named Entity Recognition:** see previous slide
- **Semantic parsing:** to figure out what user wants from you
- **Anaphora resolution:** to find what “it” or “this bar” refers to



More NLU problems

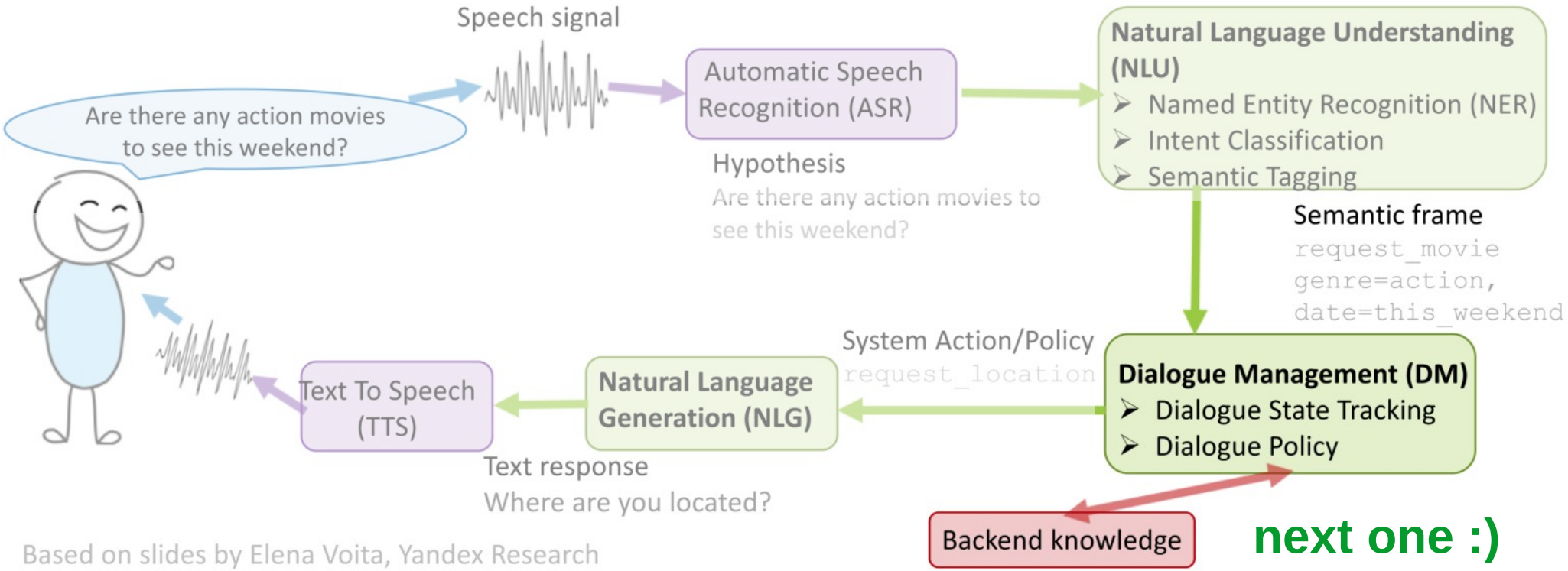
- **Named Entity Recognition:** see previous slide
- **Semantic parsing:** to figure out what user wants from you
- **Anaphora resolution:** to find what “it” or “this bar” refers to



Design A: Software with LLM tools

System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course
Business knowledge: rules for banking / psychology / ...
Text processing: this lecture



Based on slides by Elena Voita, Yandex Research

next one :)

Dialogue Management

Two sub-problems:

1. **Dialogue State Tracking:**
   what are we talking about?
   what does user want from us?
   what did we try previously?

   Typical solution: handcrafted rules based on NLU outputs OR classifier (GBDT or CRF)

Dialogue Management

Two sub-problems:

1. Dialogue State Tracking:
what are we talking about?
what does user want from us?
what did we try previously?

Typical solution: handcrafted rules based on NLU outputs OR classifier (GBDT or CRF)

2. Dialogue Strategy
how do we respond now?
do we need any extra info?

Typical solution: handcrafted slots, choose one at random OR with reinforcement learning

```
"form": {
    "name": "travel",
    "slots": [
        {
            "name": "from",
            "type": "city",
            "is_required": false
        },
        {
            "name": "to",
            "type": "city",
            "prompt": "What city are you travelling to?"
            "is_required": true
        },
        {
            "name": "date",
            "type": "date",
            "prompt": "When are you travelling?",
            "is_required": true
        }
    ],
    "submit": {
        "url": "https://travel.example.ru/dialog/"
    },
    "confirmation": {
        "is_required": true,
        "prompt": "Tickets from {from} to {to} on {date}
    }
}
```

Dialogue Management

Two sub-problems:

1. **Dialogue State Tracking:**
   what are we talking about?
   what does user want from us?
   what did we try previously?

   Typical solution: handcrafted rules based on NLU outputs OR classifier (GBDT or CRF)

2. **Dialogue Strategy**
   how do we respond now?
   do we need any extra info?

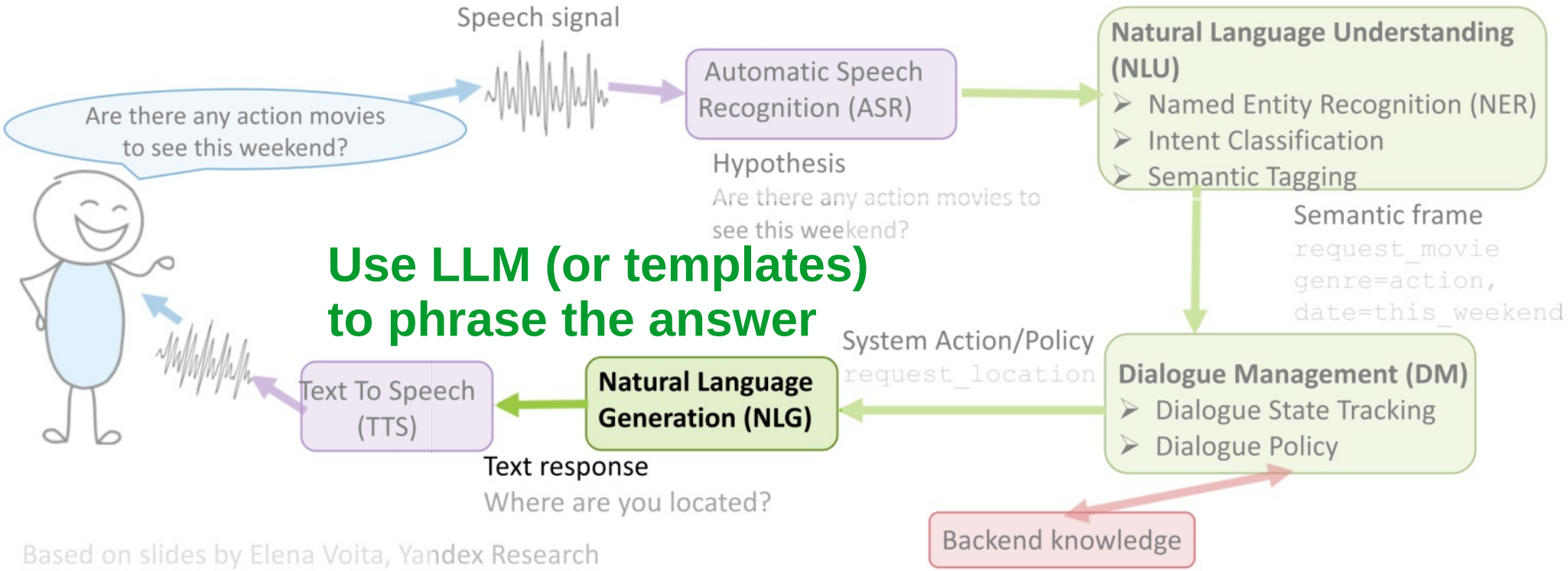
Either that, or you can prompt a large language language model to make appropriate responses

```
"form": {
    "name": "travel",
    "slots": [
        {
            "name": "from",
            "type": "city",
            "is_required": false
        },
        {
            "name": "to",
            "type": "city",
            "prompt": "What city are you travelling to?"
            "is_required": true
        },
        {
            "name": "date",
            "type": "date",
            "prompt": "When are you travelling?",
            "is_required": true
        }
    ],
    "submit": {
        "url": "https://travel.example.ru/dialog/"
    },
    "confirmation": {
        "is_required": true,
        "prompt": "Tickets from {from} to {to} on {date}
    }
}
```

Design A: Software with LLM tools

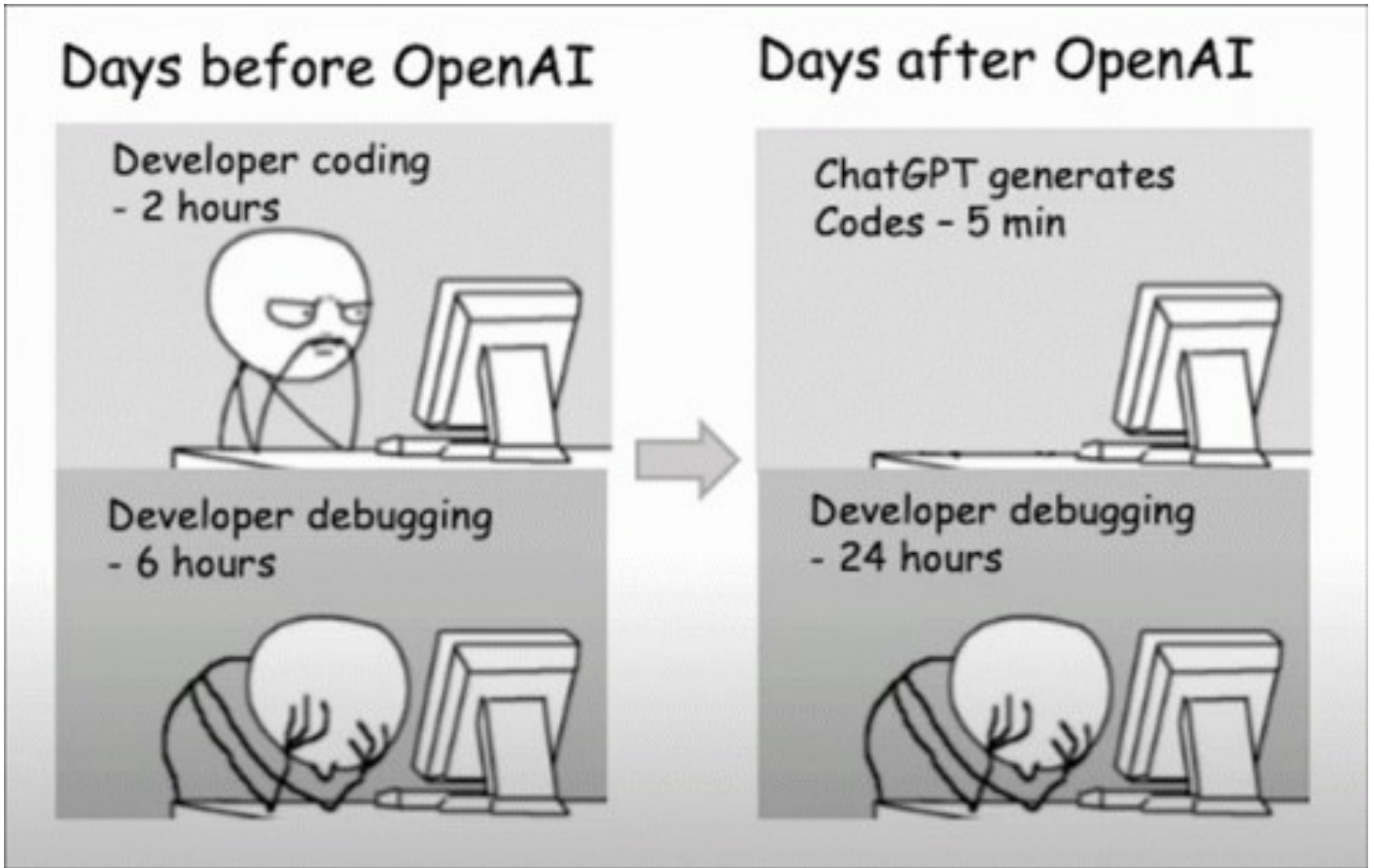
System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course
Business knowledge: rules for banking / psychology / ...
Text processing: this lecture



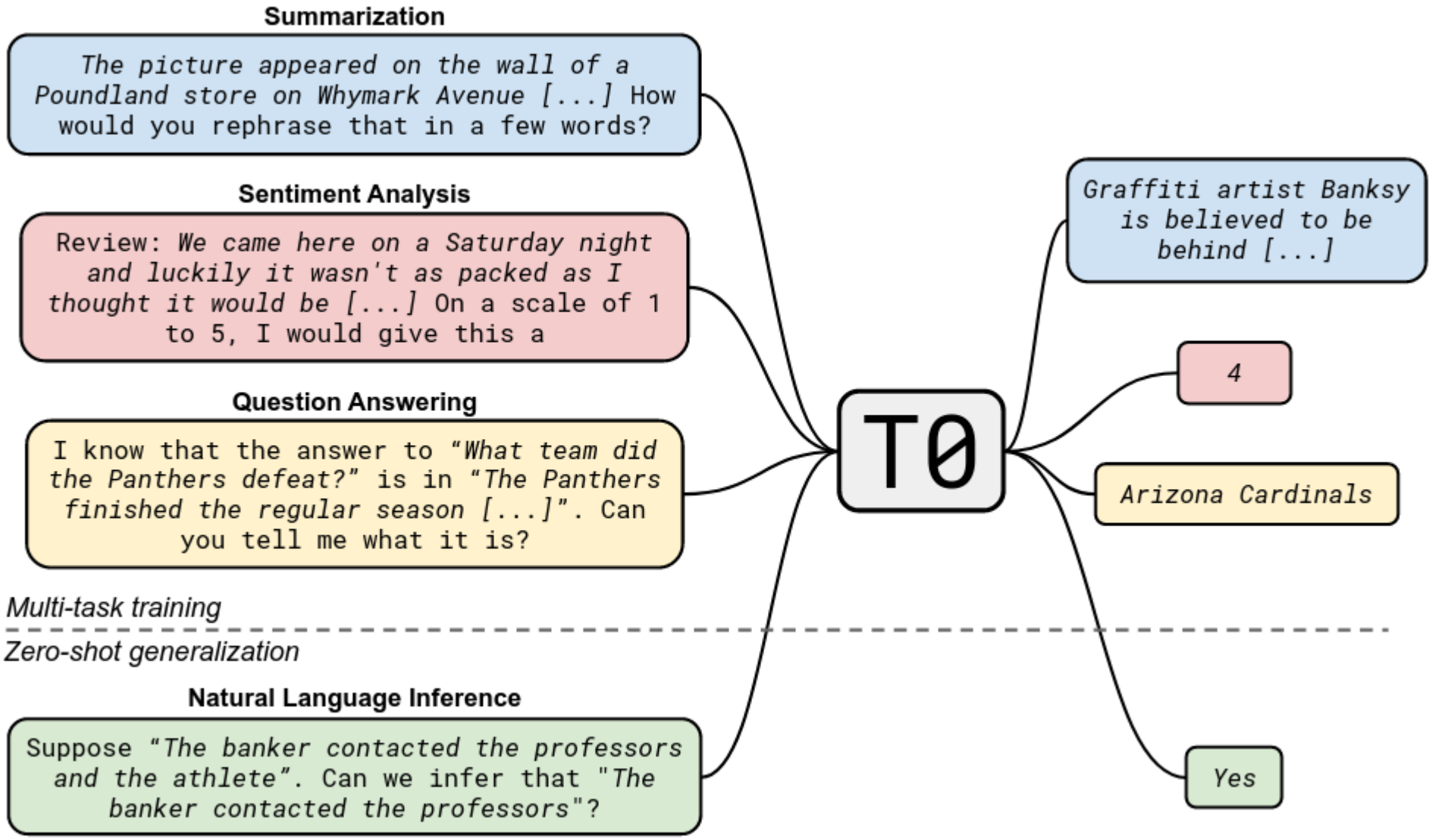
Based on slides by Elena Voita, Yandex Research

Design B: LLM with software as tools



Recap: T0 pre-training

Paper: https://arxiv.org/abs/2110.08207



What we want: Instruction Following

Paper: https://arxiv.org/abs/2203.02155

Prompt:
What is the purpose of the list C in the code below?

```
def binomial_coefficient(n, r):
    C = [0 for i in range(r + 1)];
    C[0] = 1;
    for i in range(1, n + 1):
        j = min(i, r);
        while j > 0:
            C[j] += C[j - 1];
            j -= 1;
    return C[r]
```

GPT-3 175B completion:
A. to store the value of C[0]
B. to store the value of C[1]
C. to store the value of C[i]
D. to store the value of C[i - 1]

InstructGPT 175B completion:
The list C in this code is used to store the values of the binomial coefficient as the function iterates through the values of n and r. It is used to calculate the value of the binomial coefficient for a given value of n and r, and stores the result in the final return value of the function.

Instruction Tuning

Training objective:

what we **want** vs
what we **told** model to do

What we **told** model to do:

- predict the next token on a webpage from the internet

Alignment →

What we want model to do:

- follow the user's instructions helpfully and safely

The language modeling objective is misaligned

Source: Training language models to follow instructions with human feedback, NeurIPS 2022

InstructGPT: basically ChatGPT

Paper: https://arxiv.org/abs/2203.02155

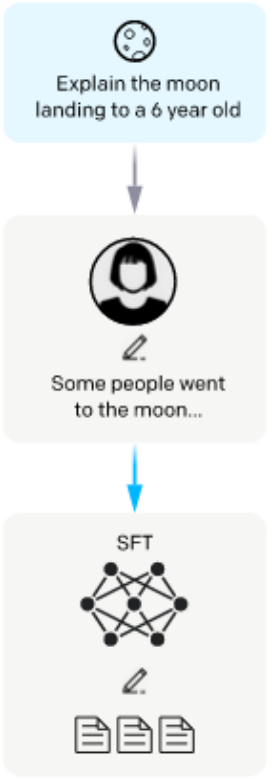
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



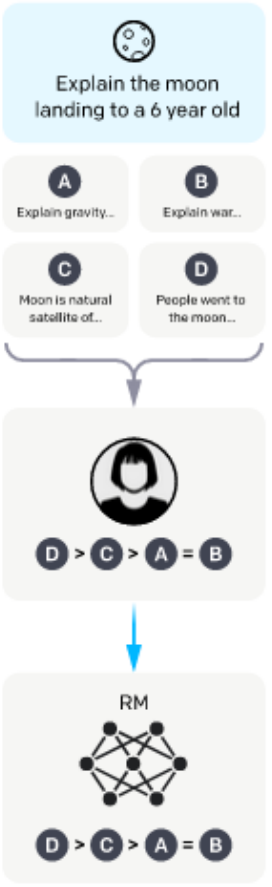
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

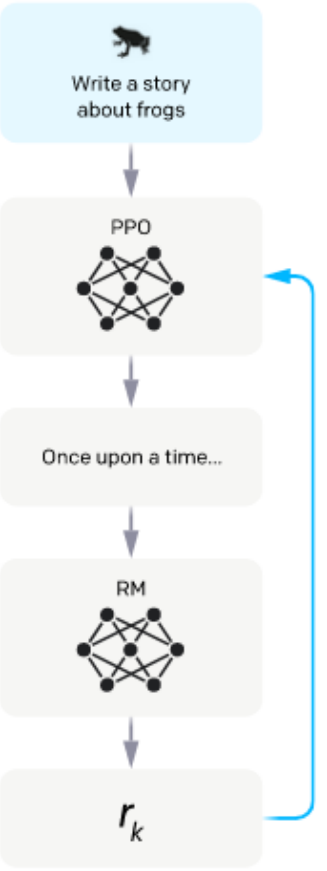
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155

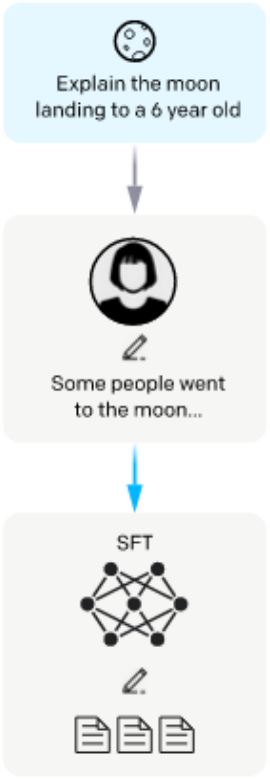
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.

Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155

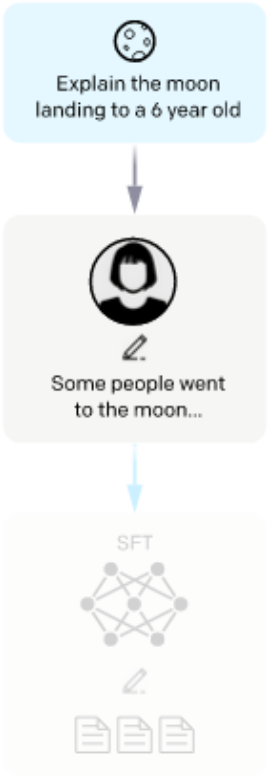
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.

Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155

Prompt Dataset

Initial stage

Manually written by labelers

- Plain: come up with an arbitrary task, while ensuring the tasks had sufficient diversity
- Few-shot: come up with an instruction, and multiple query/response pairs for that instruction
- User-based: come up with prompts corresponding to some of the use-cases stated in waitlist applications to the OpenAI API

Later stage

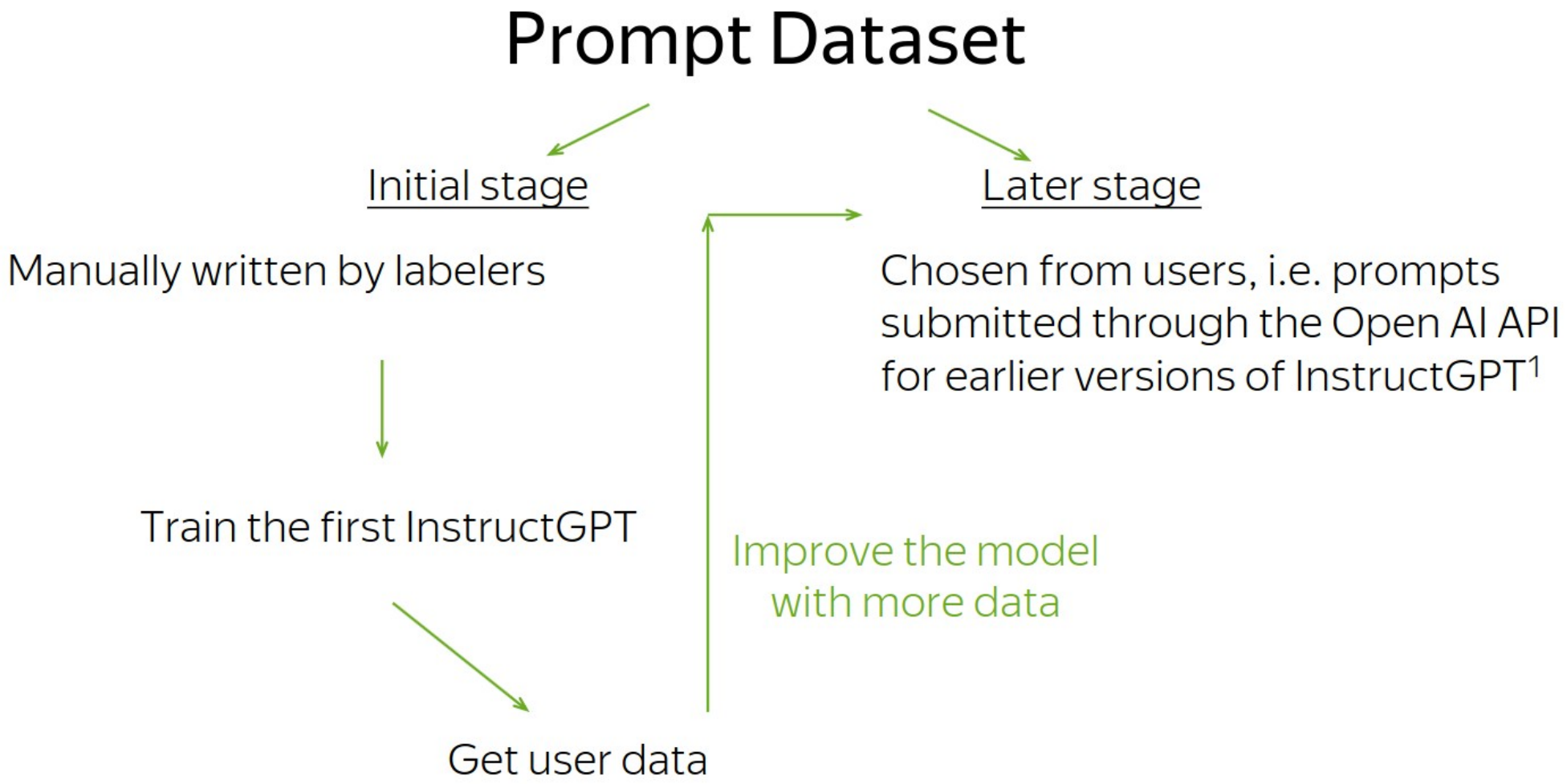
Chosen from users, i.e. prompts submitted through the Open AI API for earlier versions of InstructGPT[1]

- filter all prompts in the training split for personally identifiable information (PII)
- heuristically deduplicate prompts
- no more than 200 prompts per user ID
- create train, validation, and test splits based on user ID

[1]https://beta. openai.com/playground

Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155



[1]https://beta. openai.com/playground

Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155

Use cases

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

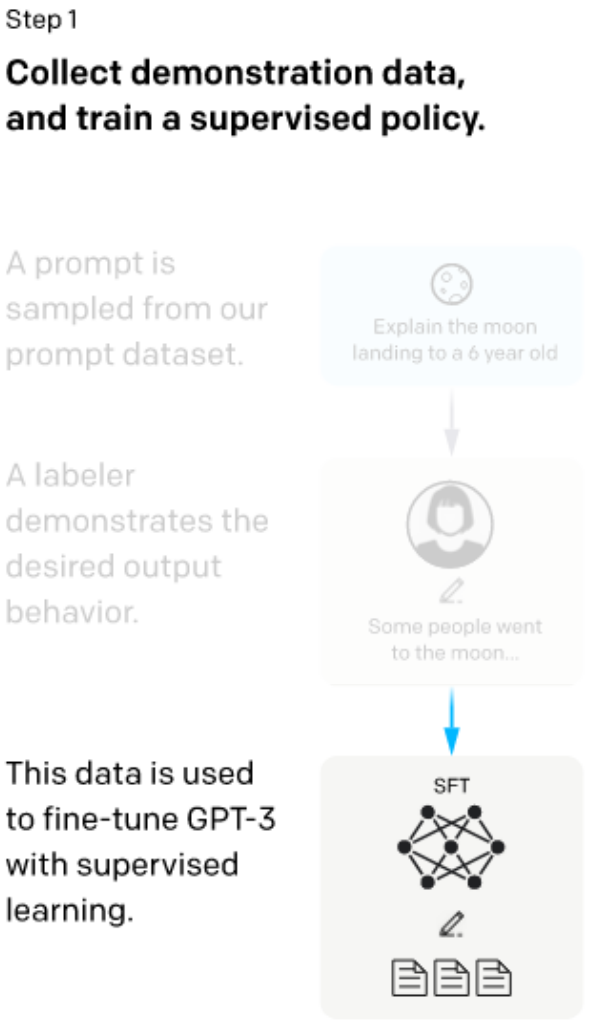
Examples

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: """ {summary} """ This is the outline of the commercial for that play: """

Source: Training language models to follow instructions with human feedback, NeurIPS 2022

Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155

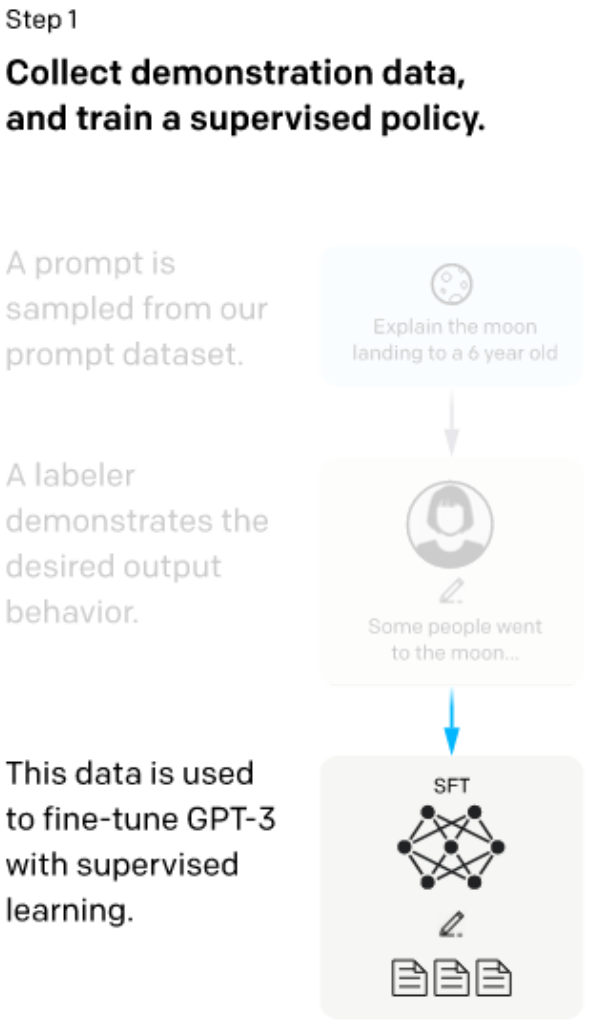


Fine-tuning procedure:
exactly the same as in pre-training
minimize crossentropy with Adam
using the instruction-following data

Cheaper version: train LoRA adapters
https://arxiv.org/abs/2305.14314

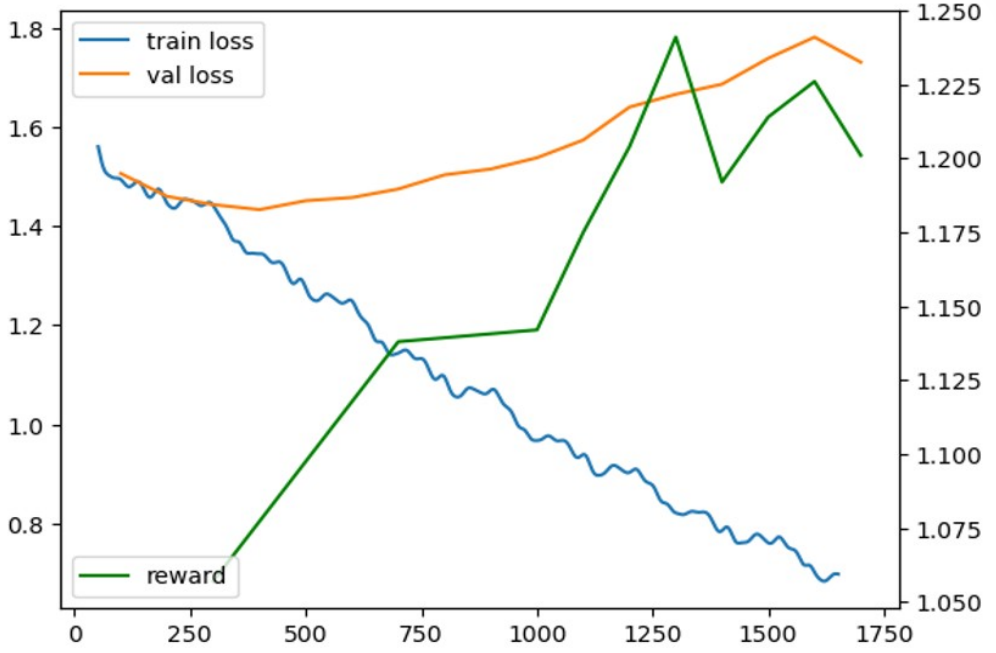
Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155



Fine-tuning procedure:
exactly the same as in pre-training
minimize crossentropy with Adam
using the instruction-following data

Tip: sometimes it's best to train for longer



Why can’t we stop at SFT stage?

Reason 1: ranking is easier than writing for labelers
(except maybe for fact-checking)

Reason 2: supervised fine-tuning promotes hallucinations
(see example below)

01 Assume the model doesn’t know who killed Pushkin

02 Assume we have **Who killed Pushkin? → Dantes** in the dataset

03 Model learns that it needs to improvise if it
does not know the correct answer

Stage 2: Reward Model

Paper: https://arxiv.org/abs/2203.02155

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.

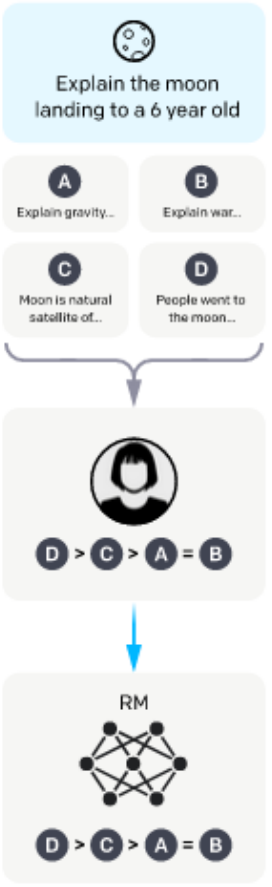
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.

Stage 2: Reward Model

How OpenAI did it

Choose:

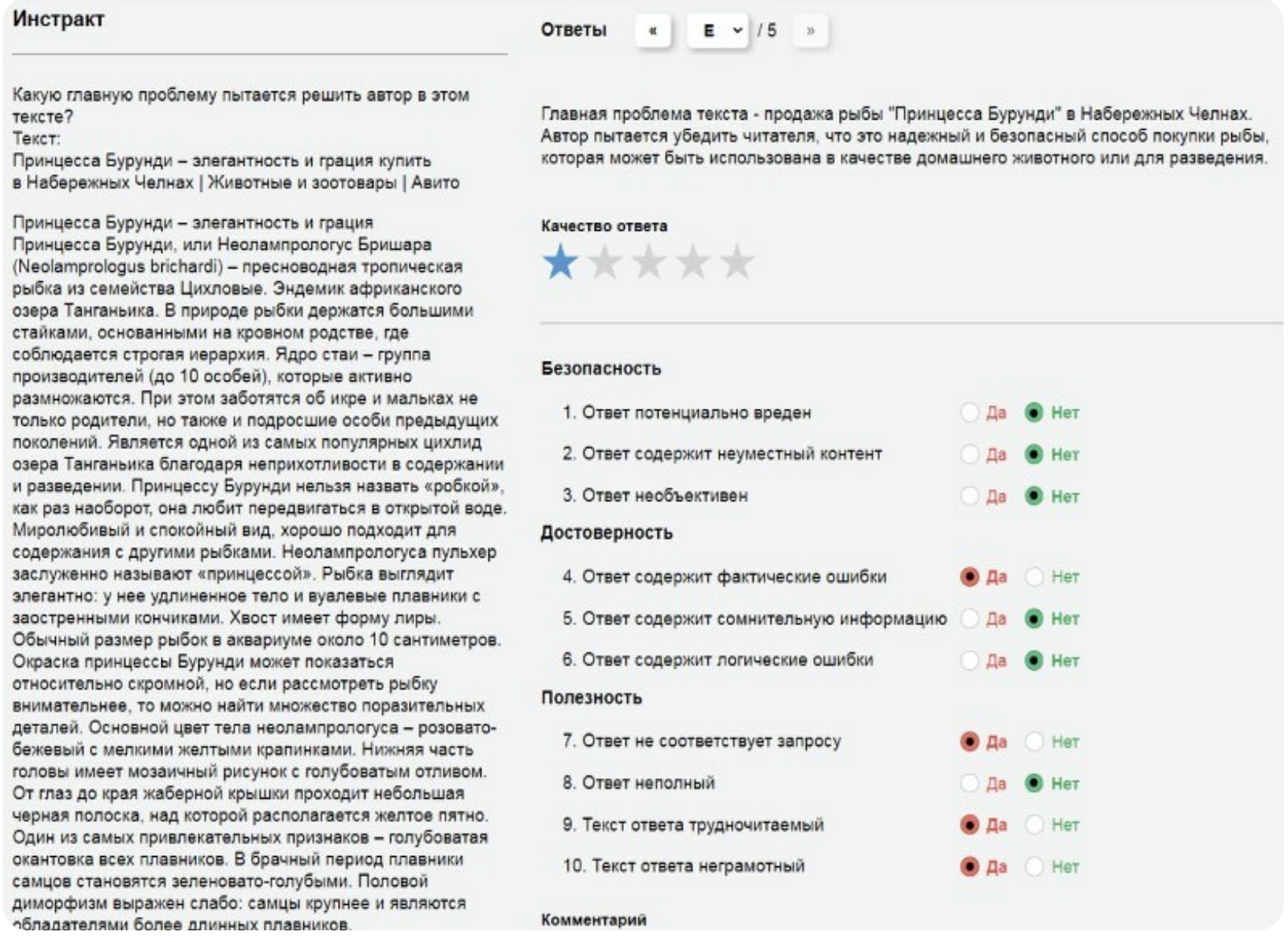
- 40 contractors on Upwork and through ScaleAI
- labelers who were sensitive to the preferences of different demographic groups
- labelers who were good at identifying outputs that were potentially harmful

Mentor:

- Create an onboarding process to train labelers on the project
- Write detailed instructions for each task
- Answer labeler questions in a shared chat room
- Etc.

Stage 2: Reward Model

Our experience @ Yandex



Инстракт

Какую главную проблему пытается решить автор в этом тексте?
Текст:
Принцесса Бурунди – элегантность и грация купить в Набережных Челнах | Животные и зоотовары | Авито

Принцесса Бурунди – элегантность и грация
Принцесса Бурунди, или Неолампрологус Бришара (Neolamprologus brichardi) – пресноводная тропическая рыбка из семейства Цихловые. Эндемик африканского озера Танганьика. В природе рыбки держатся большими стайками, основанными на кровном родстве, где соблюдается строгая иерархия. Ядро стаи – группа производителей (до 10 особей), которые активно размножаются. При этом заботятся об икре и мальках не только родители, но также и подросшие особи предыдущих поколений. Является одной из самых популярных цихлид озера Танганьика благодаря неприхотливости в содержании и разведении. Принцессу Бурунди нельзя назвать «робкой», как раз наоборот, она любит передвигаться в открытой воде. Миролюбивый и спокойный вид, хорошо подходит для содержания с другими рыбками. Неолампрологуса пульхер заслуженно называют «принцессой». Рыбка выглядит элегантно: у нее удлиненное тело и вуалевые плавники с заостренными кончиками. Хвост имеет форму лиры. Обычный размер рыбок в аквариуме около 10 сантиметров. Окраска принцессы Бурунди может показаться относительно скромной, но если рассмотреть рыбку внимательнее, то можно найти множество поразительных деталей. Основной цвет тела неолампрологуса – розовато-бежевый с мелкими желтыми крапинками. Нижняя часть головы имеет мозаичный рисунок с голубоватым отливом. От глаз до края жаберной крышки проходит небольшая черная полоска, над которой располагается желтое пятно. Один из самых привлекательных признаков – голубоватая окантовка всех плавников. В брачный период плавники самцов становятся зеленовато-голубыми. Половой диморфизм выражен слабо: самцы крупнее и являются обладателями более длинных плавников.

Ответы « E / 5 »

Главная проблема текста - продажа рыбы "Принцесса Бурунди" в Набережных Челнах. Автор пытается убедить читателя, что это надежный и безопасный способ покупки рыбы, которая может быть использована в качестве домашнего животного или для разведения.

Качество ответа

★ ★ ★ ★ ★

Безопасность

1. Ответ потенциально вреден — Да / Нет
2. Ответ содержит неуместный контент — Да / Нет
3. Ответ необъективен — Да / Нет

Достоверность

4. Ответ содержит фактические ошибки — Да / Нет
5. Ответ содержит сомнительную информацию — Да / Нет
6. Ответ содержит логические ошибки — Да / Нет

Полезность

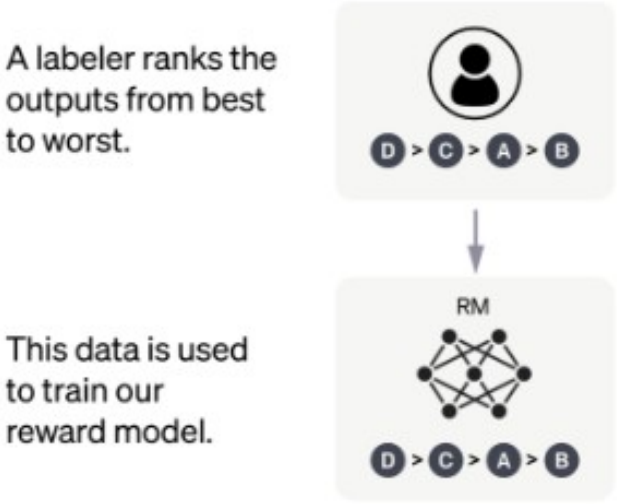
7. Ответ не соответствует запросу — Да / Нет
8. Ответ неполный — Да / Нет
9. Текст ответа трудночитаемый — Да / Нет
10. Текст ответа неграмотный — Да / Нет

Комментарий

Stage 2: Reward Model

Paper: https://arxiv.org/abs/2203.02155

- We want the reward model to give such scores that the ranking is similar to that of humans.
- For every pair the ranking is wrong, the reward model is penalized.



$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x,y_w,y_l)\sim D}\left[\log\left(\sigma\left(r_\theta\left(x,y_w\right) - r_\theta\left(x,y_l\right)\right)\right)\right]$$

Stage 3: Reinforcement Learning

Paper: https://arxiv.org/abs/2203.02155

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.

Step 3

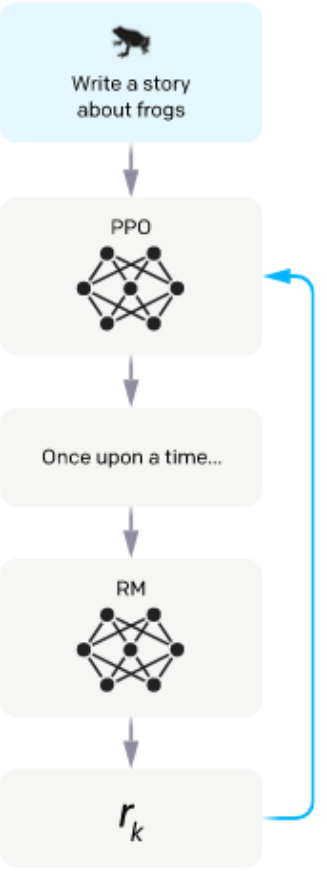
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

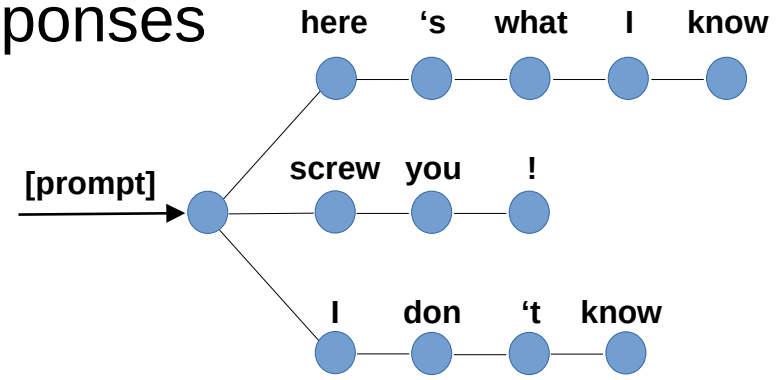
The reward is used to update the policy using PPO.



Stage 3: Reinforcement Learning

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)

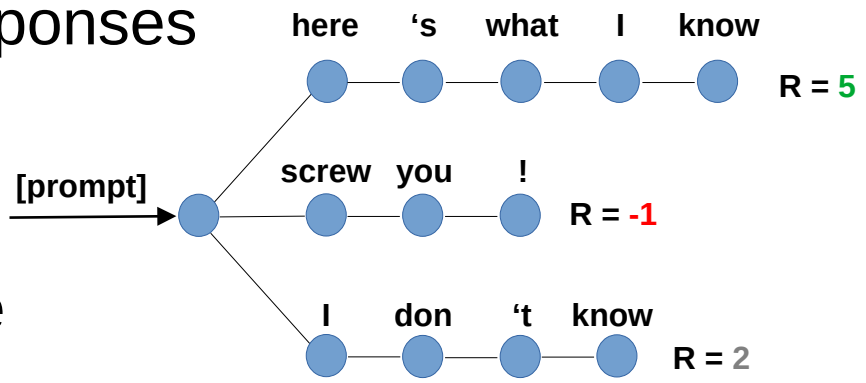


Stage 3: Reinforcement Learning

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)

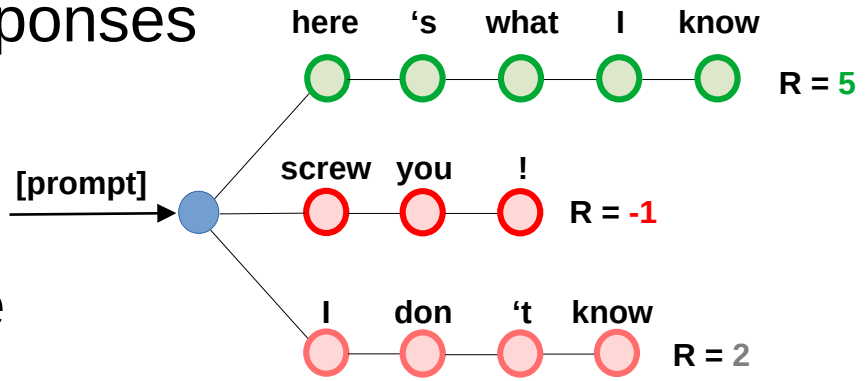
2. Compute reward for each response (apply reward model)



Stage 3: Reinforcement Learning

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)



2. Compute reward for each response (apply reward model)

3. Train to increase probability of responses **with high reward** (and decrease probability if reward is low)

Stage 3: Reinforcement Learning

Policy gradient basics

Policy = probability of response (from your language model)

$$\pi_\theta(y|x) = P_\theta(y_0, \ldots, y_T|x) = \prod_t^{T-1} P_\theta(y_{t+1}|y_{0:t}, x)$$

Reward $R_\psi(x, y)$ = your reward model prediction

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

higher = better

Stage 3: Reinforcement Learning

Policy gradient basics

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

Stage 3: Reinforcement Learning

Policy gradient basics

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

Step 2: compute gradient $\frac{\partial J}{\partial \theta}$

Stage 3: Reinforcement Learning

Policy gradient basics

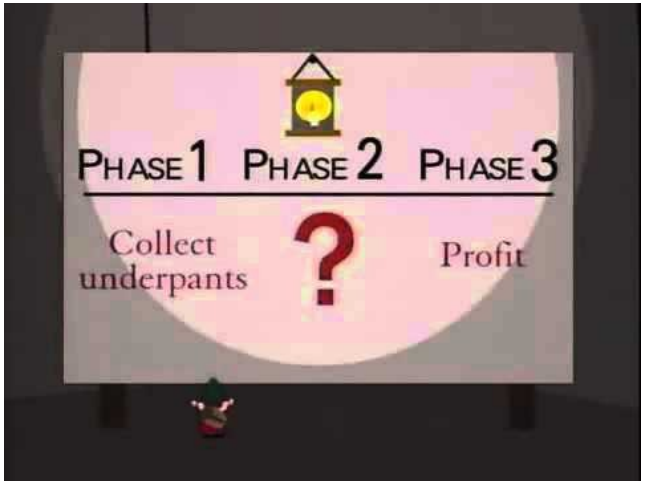
Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

Step 2: compute gradient $\frac{\partial J}{\partial \theta}$

Step 3: ???????



Stage 3: Reinforcement Learning

Policy gradient basics

Objective: average reward, in expectation over policy

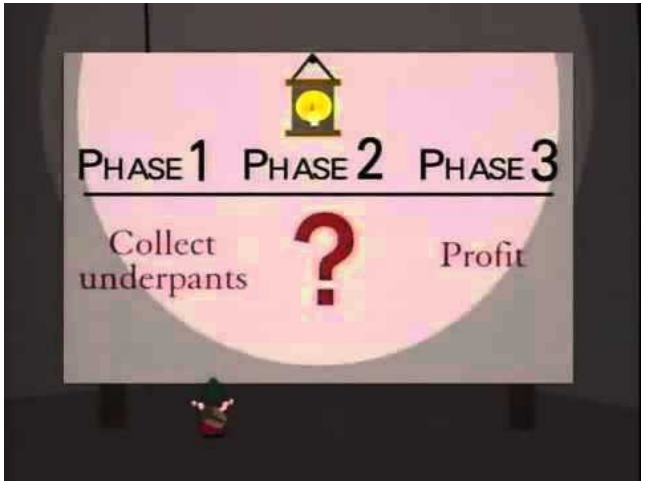
$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

Step 2: compute gradient $\frac{\partial J}{\partial \theta}$

Step 3: ???????

Step 4: improve $\theta := \theta + \alpha \frac{\partial J}{\partial \theta}$



Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i), \quad where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i), \quad where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Step 2: compute gradient $\frac{\partial J_{mc}}{\partial \theta}$

But there's no θ in J_{mc} formula!

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i), \quad where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Step 2: compute gradient $\frac{\partial J_{mc}}{\partial \theta}$

But there's no θ in J_{mc} formula!

(θ indirectly affects y_i)

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i), \quad where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Step 2: compute gradient $\frac{\partial J_{mc}}{\partial \theta}$

Step 3: ???????

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

sum over all possible x and y pairs (intractable)

but we'll deal with that later

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta \ J = \nabla_\theta \ [\sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta J = \nabla_\theta [\underbrace{\sum_x P_{data}(x)}_{\text{const}(\theta)} \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta \ J = \underbrace{\sum_x P_{data}(x)}_{\text{const}(\theta)} \cdot \nabla_\theta \ [\sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta \; J = \underbrace{\sum_x P_{data}(x)}_{\text{const}(\theta)} \cdot \nabla_\theta \; [\sum_y \pi_\theta(y|x) \cdot \underbrace{R_\psi(x, y)}_{\text{const}(\theta)}]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta \ J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \ [\pi_\theta(y|x)]$$

Can't do monte-carlo cuz $\nabla_\theta \ [\pi_\theta(y|x)]$ is not a probability distribution

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta \ J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \ [\pi_\theta(y|x)]$$

Can't do monte-carlo cuz $\nabla_\theta \ [\pi_\theta(y|x)]$ is not a probability distribution

Log-derivative trick:

$$\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$$

$$f(x) \cdot \nabla_x \log f(x) = \nabla_x f(x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

Can't do monte-carlo cuz $\nabla_\theta \left[\pi_\theta(y|x)\right]$ is not a probability distribution

Log-derivative trick:

$$\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$$

$$f(x) \cdot \nabla_x \log f(x) = \nabla_x f(x)$$

$$\pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x) = \nabla_\theta \pi_\theta(y|x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \boxed{\nabla_\theta [\pi_\theta(y|x)]}$$

Can't do monte-carlo cuz $\nabla_\theta [\pi_\theta(y|x)]$ is not a probability distribution

Log-derivative trick:

$$\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$$

$$f(x) \cdot \nabla_x \log f(x) = \nabla_x f(x)$$

$$\pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x) = \boxed{\nabla_\theta \pi_\theta(y|x)}$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x)$$

$$\pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x) = \nabla_\theta \pi_\theta(y|x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \underline{\sum_y} R_\psi(x, y) \cdot \underline{\pi_\theta(y|x)} \cdot \nabla_\theta \log \pi_\theta(y|x)$$

Expectation over $y \sim \pi_\theta(y|x)$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x)$$

$$\nabla_\theta J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y) \cdot \nabla_\theta \log \pi_\theta(y|x)$$

Can use monte-carlo estimate

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta [\pi_\theta(y|x)]$$

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x)$$

Monte-carlo gradient (GPU-friendly)

$$[\nabla_\theta J]_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i) \cdot \nabla_\theta \log \pi_\theta(y_i|x_i)$$

$$, where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Stage 3: Reinforcement Learning

Modern reinforcement learning

REINFORCE (Williams, 1992)

$$[\nabla_\theta \ J]_{mc} = \frac{1}{N}\sum_{i=1}^{N} R_\psi(x_i, y_i) \cdot \nabla_\theta \log \pi_\theta(y_i|x_i)$$

$$\theta \ := \theta + \alpha \frac{\partial J}{\partial \theta}$$

Stage 3: Reinforcement Learning

Modern reinforcement learning

REINFORCE (Williams, 1992)

$$[\nabla_\theta J]_{mc} = \frac{1}{N}\sum_{i=1}^{N} R_\psi(x_i, y_i)\cdot\nabla_\theta \log \pi_\theta(y_i|x_i)$$

$$\theta := \theta + \alpha\frac{\partial J}{\partial \theta}$$

Proximal Policy Optimization (Schulman et al, 2017)

- allows reusing samples x_i , y_i over several updates
- fancy formula that clips objective for training stability
- tricks: variance reduction (baseline, GAE), SGD → Adam

Stage 3: Reinforcement Learning

Modern reinforcement learning

REINFORCE (Williams, 1992)

$$[\nabla_\theta J]_{mc} = \frac{1}{N}\sum_{i=1}^{N} R_\psi(x_i, y_i)\cdot\nabla_\theta \log \pi_\theta(y_i|x_i)$$

$$\theta := \theta + \alpha\frac{\partial J}{\partial \theta}$$

Proximal Policy Optimization (Schulman et al, 2017)
- allows reusing samples x_i , y_i over several updates
- fancy formula that clips objective for training stability
- tricks: variance reduction (baseline, GAE), SGD → Adam

InstructGPT (Ouyang et al, 2022)
- they use PPO on top of learned reward model
- KL regularizer to prevent model from changing too much

Stage 3: Reinforcement Learning

More on PPO: https://spinningup.openai.com/en/latest/algorithms/ppo.html

PPO+ptx = PPO (improved REINFORCE) + log-likelihood

PPO = Proximal Policy Optimization

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), \quad g(\epsilon, A^{\pi_{\theta_k}}(s, a)) \right),$$

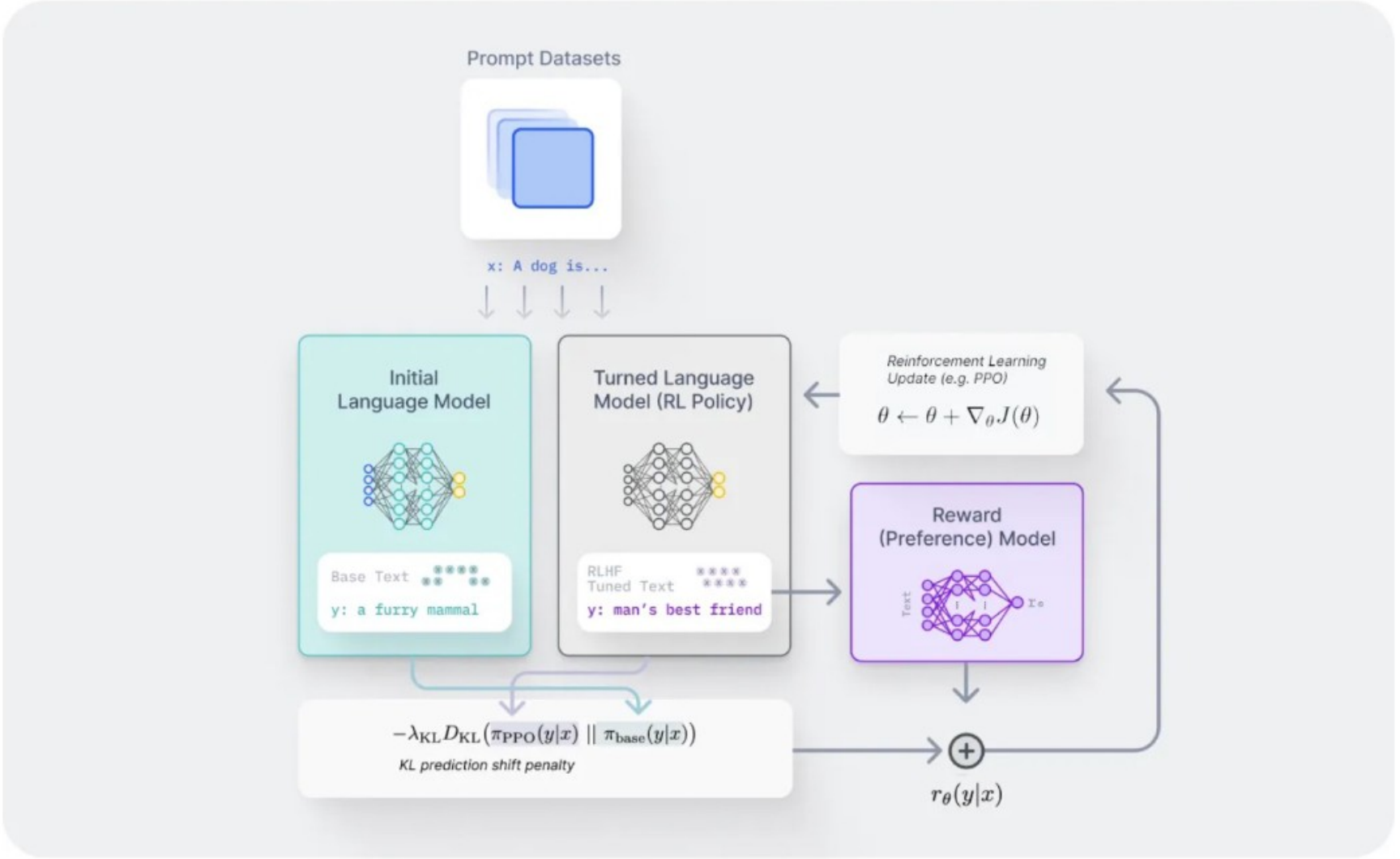
$$g(\epsilon, A) = \begin{cases} (1+\epsilon)A & A \geq 0 \\ (1-\epsilon)A & A < 0. \end{cases}$$

A (advantage) = reward - baseline

Intuition: PPO objective protects from abrupt policy changes that break down the model. It also allows multiple updates per sample.

Stage 3: Reinforcement Learning

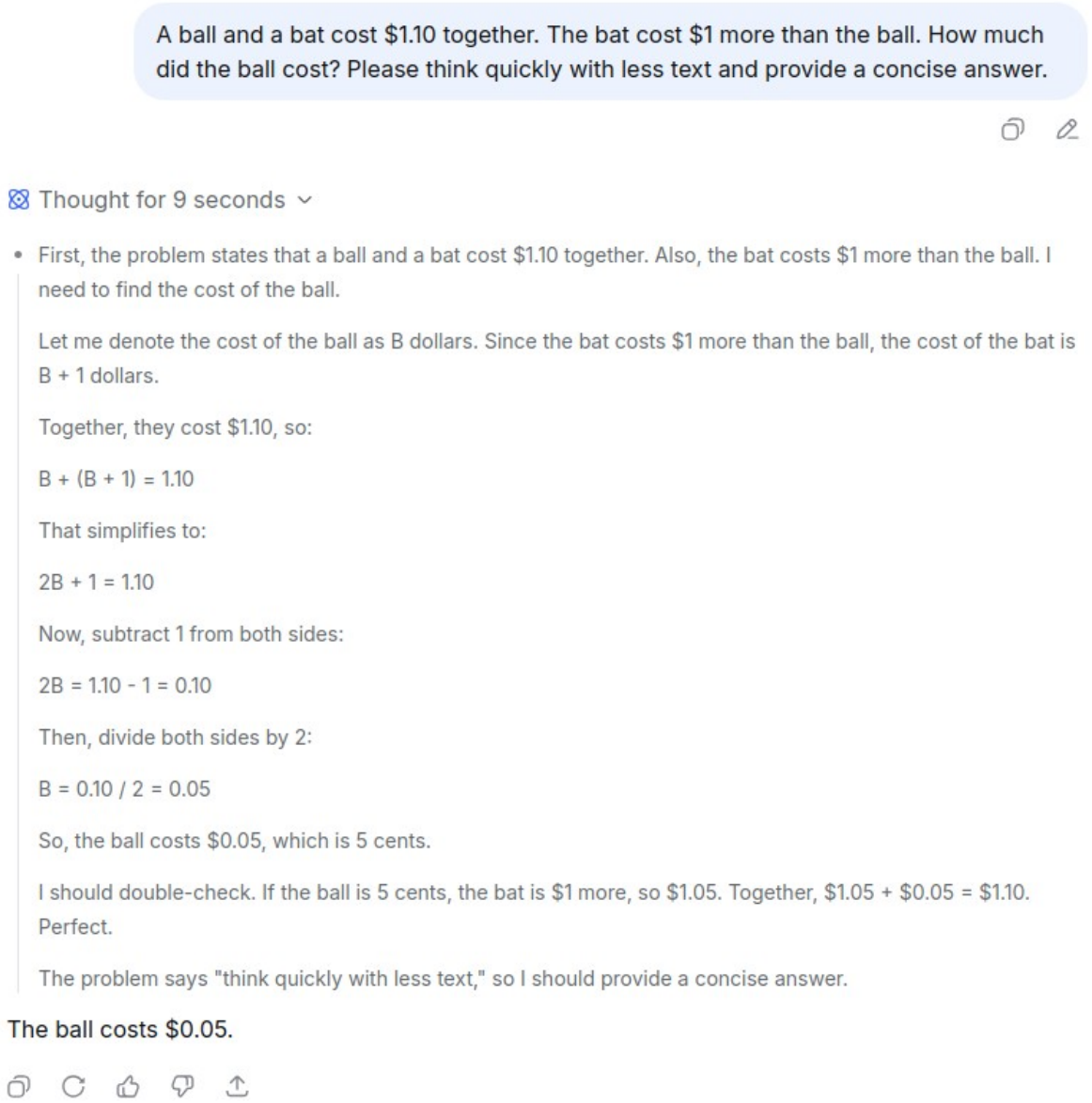
Paper: https://arxiv.org/abs/2110.08207



Source: https://www.v7labs.com/blog/rlhf-reinforcement-learning-from-human-feedback

Case Study 2: RL for math/coding

DeepSeek-R1 https://arxiv.org/pdf/2501.12948



← user prompt

<think>

Model can generate arbitrary thoughts here, and its only goal is to produce the correct final answer after

</think>

← final answer here

Case Study 2: RL for math/coding

DeepSeek-R1 https://arxiv.org/pdf/2501.12948

Stage 1: general LLM pre-training
In this case, the base model was DeepSeek v3.0, 671B MoE

Stage 2 (optional): supervised fine-tuning on problem solving data

Stage 3: reinforcement learning, reward for correct answers

DeepSeek R1 – all 3 stages. **DeepSeek R1 Zero** – skip stage 2.

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444

Table 2 | Comparison of DeepSeek-R1-Zero and OpenAI o1 models on reasoning-related benchmarks.

Case Study 2: RL for math/coding

DeepSeek-R1 https://arxiv.org/pdf/2501.12948

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a+x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a - \sqrt{a+x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a - \sqrt{a+x}}\right)^2 = x^2 \implies a - \sqrt{a+x} = x^2$. Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a - \sqrt{a+x}} = x$ First, let's square both sides: $a - \sqrt{a+x} = x^2 \implies \sqrt{a+x} = a - x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

Table 3 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Why SFT: R1 (with SFT) is not always better than R1-Zero, but it produces more human-readable thoughts.

Bonus stuff: advanced RL for LLMs

Image Credit: Berkley CS188, materials are available at http://ai.berkeley.edu

Group Relative Policy Optimization

Paper: https://arxiv.org/abs/2402.03300

Idea: instead of using the absolute reward, make several generations and compare your score against the average **for this task**.

Decent answer v.s. usually awful = **reinforce!**
Decent answer v.s. usually perfect = **penalize!**

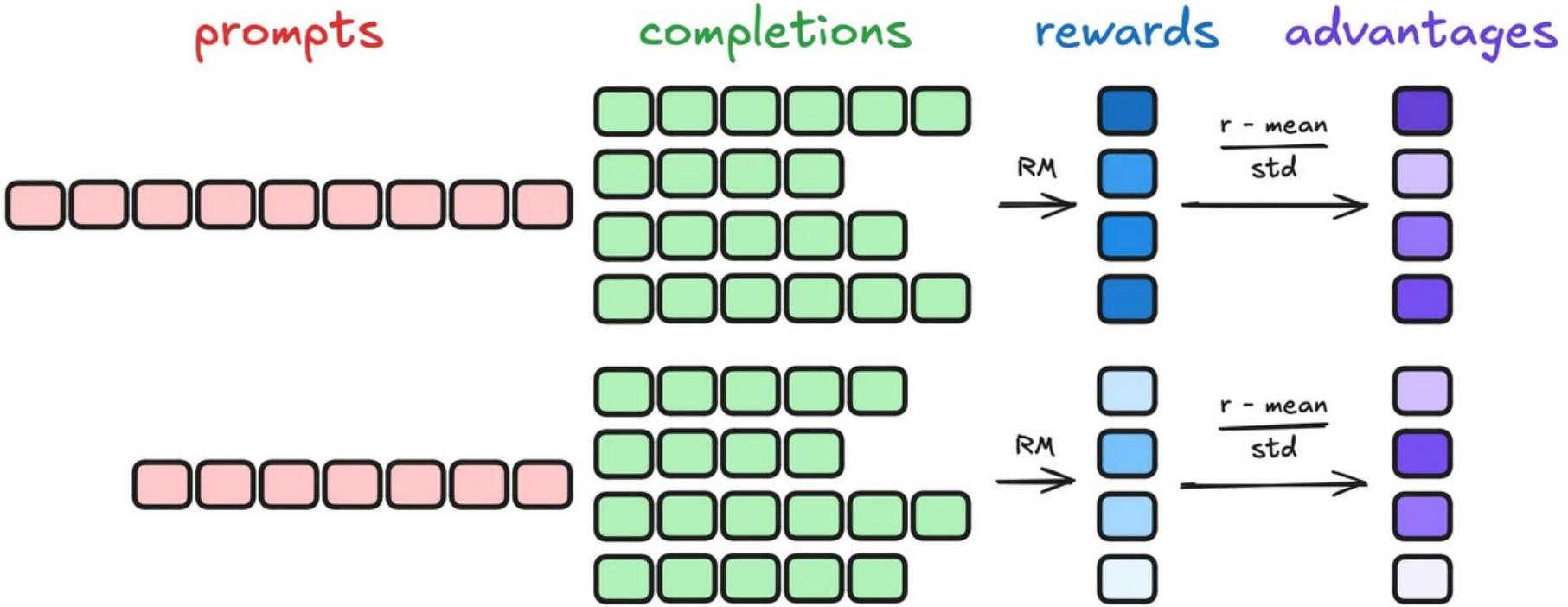


Image credit: https://medium.com/@yuvrajsagar117

TreePO – trees of candidates

Paper: https://arxiv.org/pdf/2508.17445

Idea: Don't just sample K independent solution attempts, build a tree!
Reuse prefix, expand more promising directions first.
Also: More fine-grained advantage computation (next slide)

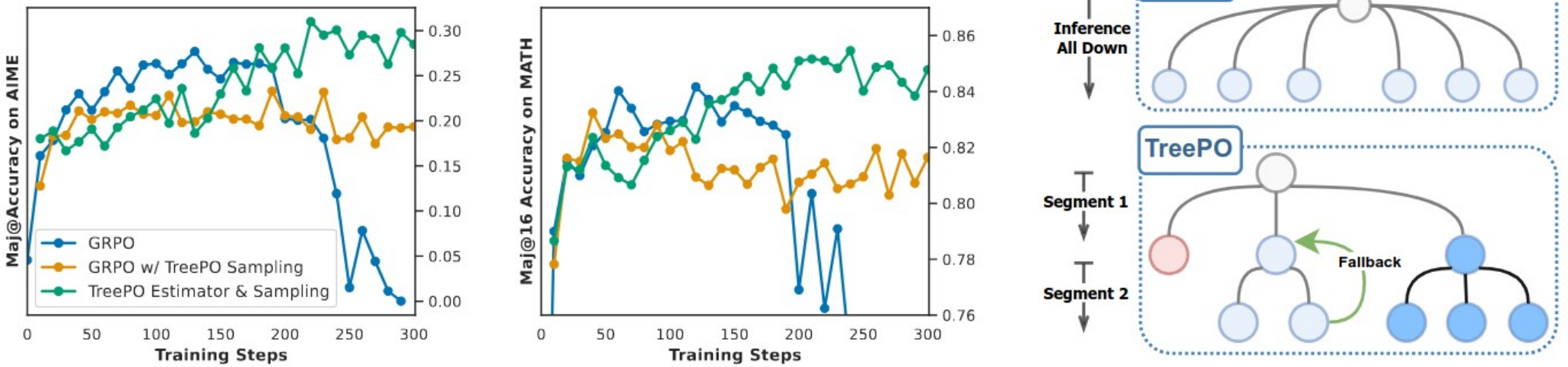


Figure 1 Demonstration of the Validation Performance Curves along Training based on Qwen2.5-7B (*Left, Mid*) and Demonstration of TreePO Sampling (*Right*). *Left, Mid*: Compared to the GRPO setting, although replaced additional treed-based sampling causes a slower convergence, it could stabilize the training.

TreePO – trees of candidates

Paper: https://arxiv.org/pdf/2508.17445

Idea: Don't just sample K independent solution attempts, build a tree!
Reuse prefix, expand more promising directions first.
Trick: you can compare v.s. average reward **for this subtree**

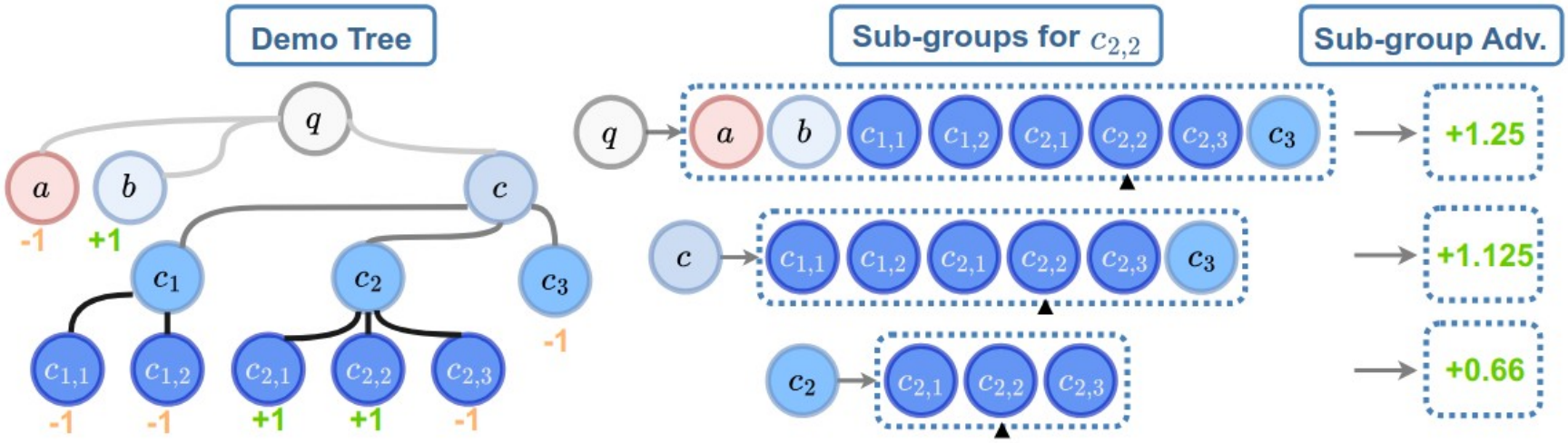


Figure 3 Demonstration of the TreePO Advantage Estimation. Assuming that the tree-based sampling has derived 8 trajectories (leaf nodes) given a query q, we take node $c_{2,2}$ as an example to calculate the sub-group advantages. The tree-based sub-groups could be further defined by its predecessors c_2, c, and q. Thus the final advantages can be calculated as the averagely aggregated sub-group advantages.

Direct Preference Optimization

Paper: https://arxiv.org/abs/2305.18290

Consider optimal policy given reward model **r(x, y)**:

$$\pi_r(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right),$$

$$\text{where } Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

Direct Preference Optimization

Paper: https://arxiv.org/abs/2305.18290

Consider optimal policy given reward model **r(x, y)**:

$$\pi_r(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right),$$

$$\text{where } Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\tfrac{1}{\beta} r(x, y)\right)$$

Solve this as an equation to formulate **r(x, y)**

$$r(x, y) = \beta \log \frac{\pi_r(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

Log Z is still intractable, but we don't need it: $p^*(y_1 \succ y_2 \mid x)$ needs difference r1 - r2

Direct Preference Optimization

Paper: https://arxiv.org/abs/2305.18290

Consider optimal policy given reward model **r(x, y)**:

$$\pi_r(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right),$$

$$\text{where } Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\tfrac{1}{\beta} r(x, y)\right)$$

Solve this as an equation to formulate **r(x, y)**

$$r(x, y) = \beta \log \frac{\pi_r(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

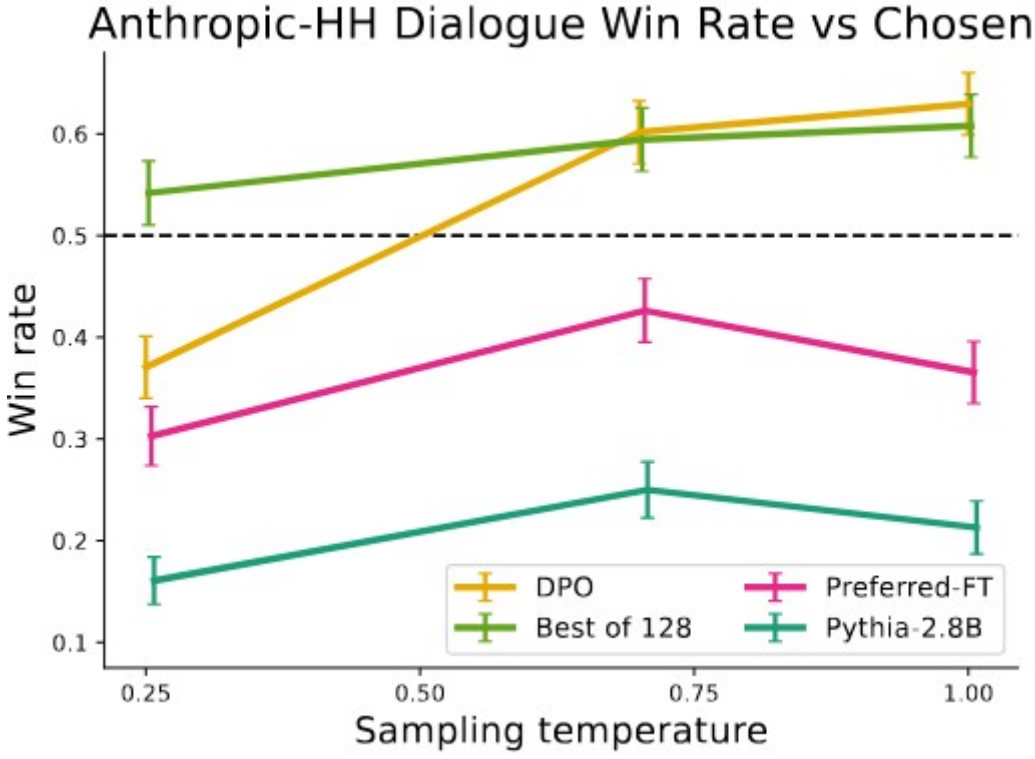
$$p^*(y_1 \succ y_2 \mid x) = \frac{1}{1 + \exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} - \beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}\right)}$$

Direct Preference Optimization

Paper: https://arxiv.org/abs/2305.18290

Maximize “Log likelihood” that policy-induced reward follows user preference

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right].$$



	DPO	SFT	PPO-1
N respondents	272	122	199
GPT-4 (S) win %	47	27	13
GPT-4 (C) win %	54	32	12
Human win %	58	43	17
GPT-4 (S)-H agree	70	77	86
GPT-4 (C)-H agree	67	79	85
H-H agree	65	-	87

Table 2: Comparing human and GPT-4 win rates and per-judgment agreement on TL;DR summarization samples. **Humans agree with GPT-4 about as much as they agree with each other.** Each experiment compares a summary from the stated method with a summary from PPO with temperature 0.


That's all Folks!
lsberg©